TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO THỰC HÀNH

HỌC PHẦN: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

ĐỀ TÀI 01: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG QUẢN LÝ THƯ VIỆN VỚI DOCKER COMPOSE, NGINX, PROMETHEUS, GRAFANA VÀ LOKI

Giảng viên hướng dẫn: Hoàng Văn Tâm
Sinh viên thực hiện: Phạm Văn Huynh
Lớp: Kỹ thuật Máy tính K23
Mã sinh viên: DTC245200357

Thái Nguyên, năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO THỰC HÀNH

HỌC PHẦN: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

ĐỀ TÀI 01: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG QUẢN LÝ THƯ VIỆN VỚI DOCKER COMPOSE, NGINX, PROMETHEUS, GRAFANA VÀ LOKI

Giảng viên hướng dẫn: Hoàng Văn Tâm
Sinh viên thực hiện: Phạm Văn Huynh
Lớp: Kỹ thuật Máy tính K23
Mã sinh viên: DTC245200357

Thái Nguyên, năm 2026

ĐẠI HỌC THÁI NGUYÊN
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

LỜI CAM ĐOAN

Em xin cam đoan báo cáo thực hành *"Triển khai và quản trị hệ thống quản lý thư viện với Docker Compose, Nginx, Prometheus, Grafana và Loki"* là kết quả do em tự thực hiện dưới sự hướng dẫn của thầy Hoàng Văn Tâm. Toàn bộ mã nguồn, file cấu hình và hình ảnh minh chứng trong báo cáo được lấy từ hệ thống em đã triển khai thực tế và được lưu trữ công khai tại kho GitHub `https://github.com/huynhpham13790-rgb/DTC245200357-LibraryManagement`.

Các tài liệu, công cụ mã nguồn mở được sử dụng trong quá trình thực hiện đều được ghi rõ trong phần tài liệu tham khảo. Nếu có gì sai trái, em xin hoàn toàn chịu trách nhiệm.

Thái Nguyên, ngày . . . tháng . . . năm 2026

SINH VIÊN THỰC HIỆN
(Ký và ghi rõ họ tên)

Phạm Văn Huynh

LỜI CẢM ƠN

Trong quá trình thực hiện báo cáo thực hành học phần *Triển khai và Quản trị Hệ thống Phần mềm*, em đã nhận được nhiều sự giúp đỡ quý báu.

Em xin gửi lời cảm ơn chân thành tới thầy **Hoàng Văn Tâm** đã tận tình giảng dạy, định hướng và góp ý cho em trong suốt quá trình học tập và thực hiện đề tài. Những kiến thức về container hóa, giám sát hệ thống, quản lý log tập trung và bảo mật hạ tầng mà thầy truyền đạt là nền tảng quan trọng để em hoàn thành bài thực hành này.

Em cũng xin cảm ơn các thầy cô Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Thông tin và Truyền thông – Đại học Thái Nguyên đã tạo điều kiện về cơ sở vật chất và môi trường học tập; cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ em.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo không tránh khỏi thiếu sót. Em rất mong nhận được ý kiến đóng góp của thầy cô để hoàn thiện hơn.

Em xin chân thành cảm ơn!

Thái Nguyên, năm 2026

Sinh viên

Phạm Văn Huynh

MỤC LỤC

LỜI CAM ĐOAN i

LỜI CẢM ƠN ii

DANH SÁCH BẢNG v

DANH SÁCH HÌNH VẼ vi

DANH MỤC TỪ VIẾT TẮT vii

MỞ ĐẦU 1

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT 3

1.1. Container hóa với Docker và Docker Compose . . . 3

1.1.1. Docker . . . 3

1.1.2. Docker Compose . . . 3

1.2. Nginx và reverse proxy . . . 3

1.3. PostgreSQL và pgAdmin . . . 4

1.4. Giám sát hệ thống với Prometheus và Grafana . . . 4

1.4.1. Prometheus . . . 4

1.4.2. Exporter . . . 4

1.4.3. Grafana . . . 4

1.5. Quản lý log tập trung với Loki và Promtail . . . 4

1.6. Hardening – thắt chặt bảo mật hệ thống . . . 5

1.7. Công nghệ xây dựng ứng dụng . . . 5

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH HỆ THỐNG 6

2.1. Yêu cầu của đề bài . . . 6

2.2. Khảo sát nghiệp vụ thư viện . . . 6

2.3. Tác nhân của hệ thống . . . 7

2.4. Yêu cầu chức năng . . . 7

2.5. Yêu cầu phi chức năng . . . 8

2.6. Kế hoạch thực hiện . . . 8

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 9

3.1. Kiến trúc tổng thể . . . 9

3.2. Thiết kế mạng . . . 10

3.3. Thiết kế cơ sở dữ liệu . . . 10

3.4. Thiết kế API . . . 11

3.5. Thiết kế xử lý mượn sách 11
3.6. Thiết kế giám sát và log 12
3.7. Thiết kế bảo mật 13

CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ **14**
4.1. Môi trường triển khai 14
4.2. Cấu trúc mã nguồn 14
4.3. Các bước triển khai 15
4.3.1. Tạo chứng chỉ SSL và biến môi trường 15
4.3.2. Khởi chạy hệ thống 15
4.3.3. Cấu hình Nginx chính 15
4.4. Kết quả ứng dụng quản lý thư viện 16
4.4.1. Đăng nhập 16
4.4.2. Trang tổng quan 16
4.4.3. Quản lý kho sách 17
4.4.4. Quản lý độc giả 18
4.4.5. Mượn, trả và gia hạn sách 19
4.4.6. Sách quá hạn và tài khoản thủ thư 22
4.5. Kết quả cơ sở dữ liệu và pgAdmin 23
4.6. Kết quả Nginx reverse proxy và HTTPS 24
4.7. Kết quả giám sát với Prometheus và Grafana 24
4.8. Kết quả log tập trung với Loki và LogQL 26
4.9. Quản lý mã nguồn trên GitHub 28
4.10. Kiểm thử 29
4.11. Đánh giá theo tiêu chí của đề bài 30

KẾT LUẬN **31**

TÀI LIỆU THAM KHẢO **33**

PHỤ LỤC: HÌNH ẢNH QUÁ TRÌNH THỰC HIỆN **34**

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN **41**

DANH SÁCH BẢNG

Bảng 2.1. Yêu cầu của đề bài và cách đáp ứng 6
Bảng 2.2. Quy định nghiệp vụ mượn – trả 7
Bảng 2.3. Yêu cầu chức năng của ứng dụng 7
Bảng 2.4. Các giai đoạn thực hiện và commit tương ứng 8
Bảng 3.1. Phân vùng mạng Docker 10
Bảng 3.2. Danh sách API của ứng dụng 11
Bảng 3.3. Các target được Prometheus thu thập 12
Bảng 3.4. Các biện pháp hardening 13
Bảng 4.1. Môi trường và phiên bản phần mềm 14
Bảng 4.2. Các truy vấn LogQL 26
Bảng 4.3. Kết quả kiểm thử chức năng và bảo mật 29
Bảng 4.4. Tự đánh giá mức độ hoàn thành 30

DANH SÁCH HÌNH VẼ

Hình 3.1. Kiến trúc tổng thể của hệ thống 9
Hình 3.2. Sơ đồ quan hệ thực thể (ERD) 10
Hình 4.1. Kết quả lệnh docker compose ps 15
Hình 4.2. Màn hình đăng nhập (trái) và thông báo khi nhập sai mật khẩu (phải) 16
Hình 4.3. Trang tổng quan của ứng dụng 16
Hình 4.4. Danh sách kho sách 17
Hình 4.5. Tìm kiếm sách theo từ khóa “linux” 18
Hình 4.6. Hộp thoại thêm sách mới 18
Hình 4.7. Danh sách độc giả 19
Hình 4.8. Lịch sử mượn sách của một độc giả 19
Hình 4.9. Từ chối cho mượn khi độc giả đang có sách quá hạn 20
Hình 4.10. Lập phiếu mượn thành công (thông báo góc phải dưới) 21
Hình 4.11. Danh sách phiếu đã trả, có phiếu bị phạt do trễ hạn 21
Hình 4.12. Danh sách sách quá hạn kèm số ngày trễ và tiền phạt tạm tính . . . 22
Hình 4.13. Quản lý tài khoản thủ thư (chỉ quản trị viên) 22
Hình 4.14. Giao diện trên màn hình điện thoại 23
Hình 4.15. pgAdmin 4 kết nối tới PostgreSQL qua reverse proxy 23
Hình 4.16. Thông tin chứng chỉ SSL tự ký 24
Hình 4.17. Kiểm tra security header và các biện pháp hardening 24
Hình 4.18. Các target trên Prometheus đều ở trạng thái UP 25
Hình 4.19. Dashboard Grafana giám sát container, web server và cơ sở dữ liệu 26
Hình 4.20. Truy vấn 1 – toàn bộ log Nginx 27
Hình 4.21. Truy vấn 2 – lọc request có mã trạng thái từ 400 trở lên 27
Hình 4.22. Truy vấn 3 – log nghiệp vụ mượn sách 28
Hình 4.23. Truy vấn 4 – các lần đăng nhập sai và bị chặn 28
Hình 4.24. Lịch sử commit của dự án 29
Hình 4.25. Repository trên GitHub 29
Hình PL.1. Tạo cấu trúc thư mục dự án và các file mã nguồn ứng dụng 34
Hình PL.2. Tạo chứng chỉ SSL tự ký bằng OpenSSL 34
Hình PL.3. Khởi tạo Git repository và commit đầu tiên 35
Hình PL.4. Kiểm tra cấu hình hệ thống bằng lệnh docker compose config . . . 35
Hình PL.5. Kiểm tra trạng thái container và đọc log Nginx khi gỡ lỗi kết nối tới pgAdmin 36
Hình PL.6. Giao diện pgAdmin theo dõi hoạt động của PostgreSQL 36
Hình PL.7. Log khởi động Grafana và commit tích hợp Prometheus, Grafana . 37
Hình PL.8. Kiểm tra các container và gọi thử API ứng dụng bằng curl 37

Hình PL.9. Truy vấn log Nginx trên Grafana Explore (lần chạy đầu) 38
Hình PL.10. Lọc log theo mã trạng thái trên Grafana Explore (lần chạy đầu) . . 38
Hình PL.11. Truy vấn dạng metric từ log trên Grafana Explore 39
Hình PL.12. Kiểm thử API và commit triển khai Loki, Promtail 39
Hình PL.13. Giao diện phiên bản đầu tiên của ứng dụng 40
Hình PL.14. Hoàn thiện cấu hình và đẩy mã nguồn lên GitHub 40

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	**Diễn giải**
API	Application Programming Interface – giao diện lập trình ứng dụng
CNTT	Công nghệ thông tin
CRUD	Create, Read, Update, Delete – các thao tác thêm, đọc, sửa, xóa dữ liệu
CSDL	Cơ sở dữ liệu
CSRF	Cross-Site Request Forgery – tấn công giả mạo yêu cầu
DB	Database – cơ sở dữ liệu
HSTS	HTTP Strict Transport Security
HTTP(S)	HyperText Transfer Protocol (Secure)
ISBN	International Standard Book Number – mã số sách tiêu chuẩn quốc tế
LogQL	Log Query Language – ngôn ngữ truy vấn log của Loki
NXB	Nhà xuất bản
PromQL	Prometheus Query Language
RAM	Random Access Memory
REST	Representational State Transfer
SPA	Single Page Application – ứng dụng web một trang
SQL	Structured Query Language
SSL/TLS	Secure Sockets Layer / Transport Layer Security
UI	User Interface – giao diện người dùng
XSS	Cross-Site Scripting – tấn công chèn mã kịch bản

MỞ ĐẦU

Lý do chọn đề tài

Thư viện là đơn vị không thể thiếu trong các trường đại học. Việc quản lý thủ công bằng sổ sách hoặc bảng tính khiến công tác tra cứu sách, theo dõi độc giả, lập phiếu mượn – trả và nhắc nhở sách quá hạn tốn nhiều thời gian và dễ sai sót. Một phần mềm quản lý thư viện chạy trên nền web giúp tự động hóa các nghiệp vụ này.

Tuy nhiên, để một phần mềm vận hành ổn định trong thực tế, ngoài việc lập trình chức năng, người quản trị còn phải giải quyết bài toán *triển khai và vận hành*: đóng gói ứng dụng, kết nối cơ sở dữ liệu, công bố dịch vụ an toàn qua HTTPS, giám sát tài nguyên, thu thập log tập trung để truy vết sự cố và thắt chặt bảo mật. Đây chính là nội dung trọng tâm của học phần Triển khai và Quản trị Hệ thống Phần mềm. Vì vậy em lựa chọn **Đề tài 01 – Hệ thống Quản lý Thư viện** để vừa xây dựng ứng dụng, vừa thực hành đầy đủ quy trình triển khai, giám sát và bảo mật hệ thống.

Mục tiêu

- Xây dựng ứng dụng web quản lý thư viện gồm: quản lý kho sách, quản lý độc giả, mượn – trả – gia hạn sách, theo dõi sách quá hạn, thống kê và phân quyền tài khoản thủ thư.
- Triển khai toàn bộ hệ thống bằng Docker Compose với PostgreSQL và công cụ quản trị pgAdmin.
- Cấu hình Nginx làm reverse proxy có HTTPS tự ký và các HTTP security header.
- Tích hợp Prometheus, Grafana cùng các exporter để giám sát container, web server và cơ sở dữ liệu.
- Triển khai hệ thống log tập trung Loki + Promtail và truy vấn bằng LogQL.
- Áp dụng các biện pháp hardening: non-root container, cô lập mạng, mật khẩu mạnh, hạn chế quyền, giới hạn tần suất truy cập.
- Quản lý mã nguồn trên GitHub với các commit rõ ràng và README hướng dẫn chạy.

Đối tượng và phạm vi

Đối tượng: các công nghệ container hóa (Docker, Docker Compose), reverse proxy (Nginx), giám sát (Prometheus, Grafana, cAdvisor, Postgres Exporter, Nginx Exporter), log tập trung (Loki, Promtail) và nghiệp vụ quản lý thư viện.

Phạm vi: hệ thống được triển khai trên một máy chủ Linux (Ubuntu) dùng Docker Engine; người dùng ứng dụng là cán bộ thủ thư; chứng chỉ SSL là chứng chỉ tự ký phục vụ môi trường học tập.

Phương pháp thực hiện

- Nghiên cứu tài liệu chính thức của các công nghệ sử dụng.

- Phân tích yêu cầu của đề bài và nghiệp vụ thư viện, thiết kế kiến trúc, cơ sở dữ liệu và API.
- Triển khai theo từng giai đoạn, mỗi giai đoạn tương ứng một commit trên GitHub.
- Kiểm thử chức năng, kiểm tra cấu hình bảo mật và thu thập minh chứng bằng ảnh chụp màn hình.

Cấu trúc báo cáo

Ngoài phần mở đầu và kết luận, báo cáo gồm 4 chương:

- **Chương 1 – Cơ sở lý thuyết:** trình bày các công nghệ được sử dụng.
- **Chương 2 – Khảo sát và phân tích hệ thống:** phân tích yêu cầu đề bài, nghiệp vụ thư viện, yêu cầu chức năng và phi chức năng.
- **Chương 3 – Thiết kế hệ thống:** kiến trúc tổng thể, thiết kế mạng, cơ sở dữ liệu, API, giám sát, log và bảo mật.
- **Chương 4 – Triển khai và kết quả:** quá trình cài đặt, kết quả chạy thực tế, minh chứng và đánh giá theo tiêu chí của đề bài.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

Chương này trình bày các khái niệm và công nghệ nền tảng được sử dụng để xây dựng, triển khai và vận hành hệ thống quản lý thư viện.

1.1. Container hóa với Docker và Docker Compose

1.1.1. Docker

Docker là nền tảng ảo hóa ở mức hệ điều hành, cho phép đóng gói ứng dụng cùng toàn bộ thư viện phụ thuộc thành một *image* và chạy dưới dạng *container* cô lập [1]. Khác với máy ảo, các container dùng chung nhân Linux của máy chủ nên khởi động nhanh, tốn ít tài nguyên. Các khái niệm chính:

- **Image:** bản mẫu chỉ đọc được xây dựng từ `Dockerfile`, gồm nhiều lớp (layer).
- **Container:** một thực thể đang chạy của image, có hệ thống file, tiến trình và mạng riêng.
- **Volume:** vùng lưu trữ bền vững nằm ngoài vòng đời container, dùng cho dữ liệu cơ sở dữ liệu, Grafana, Loki.
- **Network:** mạng ảo (thường là *bridge*) giúp các container giao tiếp với nhau qua tên dịch vụ.

1.1.2. Docker Compose

Docker Compose cho phép mô tả một hệ thống gồm nhiều container trong một file YAML duy nhất (`docker-compose.yml`): image, biến môi trường, volume, mạng, cổng và thứ tự phụ thuộc [2]. Toàn bộ hệ thống được khởi chạy bằng một lệnh `docker compose up -d`, giúp việc triển khai lặp lại được và dễ quản lý phiên bản bằng Git.

1.2. Nginx và reverse proxy

Nginx là web server hiệu năng cao, thường được dùng làm *reverse proxy* – điểm tiếp nhận duy nhất của mọi yêu cầu từ người dùng rồi chuyển tiếp tới các dịch vụ phía sau [3]. Việc đặt Nginx phía trước mang lại các lợi ích:

- Chỉ một container mở cổng ra ngoài (80/443), các dịch vụ khác được ẩn trong mạng nội bộ.
- Kết thúc mã hóa TLS (TLS termination) tập trung, các dịch vụ nội bộ không cần tự cấu hình HTTPS.
- Định tuyến theo đường dẫn: `/` tới ứng dụng web, `/pgadmin/` tới pgAdmin, `/grafana/` tới Grafana.
- Bổ sung các HTTP security header, giới hạn tần suất truy cập (rate limiting) và ghi access log thống nhất.

HTTPS và chứng chỉ tự ký: HTTPS dùng giao thức TLS để mã hóa dữ liệu giữa trình duyệt và máy chủ. Trong môi trường học tập, chứng chỉ được tạo bằng OpenSSL

(RSA 2048 bit, hạn 365 ngày) thay vì mua từ tổ chức cấp chứng chỉ; trình duyệt sẽ cảnh báo "Not secure" nhưng kênh truyền vẫn được mã hóa.

1.3. PostgreSQL và pgAdmin

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, hỗ trợ đầy đủ giao dịch ACID, khóa dòng (`SELECT ... FOR UPDATE`), ràng buộc toàn vẹn và nhiều kiểu dữ liệu [4]. Hệ thống dùng image `postgres:16-alpine`. pgAdmin 4 là công cụ quản trị PostgreSQL qua giao diện web, cho phép xem cấu trúc bảng, chạy câu lệnh SQL và theo dõi hoạt động của máy chủ cơ sở dữ liệu.

1.4. Giám sát hệ thống với Prometheus và Grafana

1.4.1. Prometheus

Prometheus là hệ thống giám sát dựa trên chuỗi thời gian (time series) [5]. Prometheus hoạt động theo mô hình *pull*: định kỳ (ở hệ thống này là 5 giây) gửi yêu cầu HTTP tới endpoint `/metrics` của các *target* để thu thập số liệu, lưu lại và cho phép truy vấn bằng ngôn ngữ PromQL. Ví dụ, tốc độ request mỗi giây của Nginx được tính bằng biểu thức:

```
rate(nginx_http_requests_total[1m])
```

1.4.2. Exporter

Exporter là thành phần trung gian chuyển số liệu của một phần mềm sang định dạng Prometheus:

- **cAdvisor:** thu thập CPU, RAM, mạng, ổ đĩa của từng container.
- **Postgres Exporter:** số kết nối, số giao dịch commit/rollback, kích thước cơ sở dữ liệu...
- **Nginx Exporter:** đọc trang `stub_status` của Nginx để lấy số kết nối và số request.
- **prom-client:** thư viện Node.js giúp ứng dụng tự xuất metrics (request theo route, số lần đăng nhập, số thao tác mượn/trả).

1.4.3. Grafana

Grafana là nền tảng trực quan hóa dữ liệu, kết nối tới nhiều nguồn dữ liệu (Prometheus, Loki...) và hiển thị dưới dạng dashboard [6]. Grafana hỗ trợ *provisioning* – khai báo sẵn datasource và dashboard bằng file cấu hình, nhờ đó dashboard được quản lý trong Git và tự động xuất hiện khi khởi động hệ thống.

1.5. Quản lý log tập trung với Loki và Promtail

Loki là hệ thống lưu trữ log do Grafana Labs phát triển, được thiết kế theo triết lý "giống Prometheus nhưng dành cho log" [7]. Thay vì đánh chỉ mục toàn văn như Elasticsearch, Loki chỉ đánh chỉ mục các *nhãn* (label) nên rất nhẹ. Promtail là agent đọc file log, gắn nhãn (ví dụ `job="nginx"`) và đẩy về Loki.

LogQL là ngôn ngữ truy vấn log của Loki, gồm hai phần: *bộ chọn luồng log* theo nhãn và *pipeline* xử lý. Ví dụ:

- `{job="nginx"}`: lấy toàn bộ log của Nginx.
- `{job="nginx"} |= "/api/borrow"`: lọc các dòng có chứa chuỗi `/api/borrow`.
- `{job="nginx"} | json | status >= 400`: phân tích log dạng JSON rồi lọc theo trường `status`.
- Đếm số log lỗi mỗi phút (metric query):
  `sum(count_over_time({job="nginx"} | json | status >= 400 [1m]))`

1.6. Hardening – thắt chặt bảo mật hệ thống

Hardening là quá trình giảm bề mặt tấn công của hệ thống bằng cách loại bỏ quyền và thành phần không cần thiết [8, 9]. Các nguyên tắc chính được áp dụng trong đề tài:

- **Đặc quyền tối thiểu (least privilege):** container chạy bằng user thường (non-root), bỏ các Linux capability không cần thiết, bật `no-new-privileges`, hệ thống file chỉ đọc.
- **Phân vùng mạng (network segmentation):** chia mạng frontend, backend, monitoring; mạng cơ sở dữ liệu không có đường ra Internet.
- **Quản lý bí mật:** mật khẩu mạnh, lưu trong file `.env` không đưa lên Git.
- **Bảo mật tầng ứng dụng web:** HTTPS, security header, chống SQL injection, XSS, dò mật khẩu (brute force) theo khuyến nghị OWASP.

1.7. Công nghệ xây dựng ứng dụng

Ứng dụng quản lý thư viện được viết bằng **Node.js** với framework **Express**, kết nối PostgreSQL qua thư viện `pg`. Giao diện là ứng dụng web một trang (SPA) dùng HTML, JavaScript thuần và **Bootstrap 5**, giao tiếp với máy chủ qua REST API định dạng JSON.

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH HỆ THỐNG

Chương này phân tích yêu cầu của đề bài, khảo sát nghiệp vụ thư viện và xác định các yêu cầu chức năng, phi chức năng của hệ thống.

Việc phân tích được thực hiện theo quy trình kỹ nghệ yêu cầu gồm khảo sát, đặc tả và kiểm chứng yêu cầu [10].

2.1. Yêu cầu của đề bài

Đề tài 01 yêu cầu triển khai một ứng dụng web quản lý thư viện (mượn/trả sách, quản lý độc giả, sách) sử dụng PostgreSQL và pgAdmin, cùng các yêu cầu kỹ thuật chung được tổng hợp trong Bảng 2.1.

Bảng 2.1. *Yêu cầu của đề bài và cách đáp ứng*

STT	Yêu cầu	Cách đáp ứng trong đề tài
1	Quản lý mã nguồn trên GitHub, commit rõ ràng, có README	Repository `DTC245200357-LibraryManagement`, các commit theo từng giai đoạn, README hướng dẫn chạy
2	Triển khai ứng dụng web + DB + công cụ quản lý DB	Ứng dụng Node.js/Express, PostgreSQL 16, pgAdmin 4
3	Nginx reverse proxy, HTTPS tự ký hoặc security header	Nginx với chứng chỉ tự ký, chuyển hướng HTTP sang HTTPS, 5 security header
4	Prometheus + Grafana giám sát container, web server, DB	cAdvisor, Nginx Exporter, Postgres Exporter, metrics ứng dụng; dashboard Grafana được provision sẵn
5	Loki + Promtail, truy vấn LogQL (2–3 query)	Promtail thu thập access/error log Nginx; 4 truy vấn LogQL minh họa
6	Hardening (non-root, cô lập mạng, mật khẩu mạnh, hạn chế quyền, security header)	Áp dụng 10 biện pháp, trình bày tại mục 3.7
7	Báo cáo tối thiểu 10 trang, có hình minh họa	Báo cáo này

2.2. Khảo sát nghiệp vụ thư viện

Qua tìm hiểu hoạt động của thư viện trường đại học, quy trình nghiệp vụ chính gồm:

1. **Bổ sung sách:** thủ thư nhập thông tin đầu sách (tên, tác giả, ISBN, thể loại, nhà xuất bản, năm xuất bản), số lượng bản và vị trí kệ.

2. **Cấp thẻ độc giả:** độc giả là sinh viên, giảng viên, cán bộ hoặc khách; mỗi người có một mã thẻ duy nhất và thời hạn sử dụng thẻ.
3. **Cho mượn sách:** thủ thư kiểm tra thẻ còn hạn, độc giả không nợ sách quá hạn, chưa vượt số sách tối đa và sách còn bản; sau đó lập phiếu mượn ghi rõ hạn trả.
4. **Gia hạn:** độc giả được gia hạn khi phiếu chưa quá hạn.
5. **Nhận trả sách:** thủ thư ghi nhận ngày trả; nếu trễ hạn thì thu tiền phạt theo số ngày trễ.
6. **Theo dõi, báo cáo:** thống kê số sách, số lượt mượn, danh sách sách quá hạn để nhắc nhở.

Các quy định nghiệp vụ được áp dụng trong hệ thống được tổng hợp trong Bảng 2.2.

Bảng 2.2. *Quy định nghiệp vụ mượn – trả*

STT	Quy định	Giá trị
1	Số sách tối đa mỗi độc giả mượn cùng lúc	5 cuốn
2	Thời hạn mượn	7, 14 (mặc định) hoặc 30 ngày
3	Gia hạn	Cộng thêm 7 ngày, tối đa 1 lần, chỉ khi chưa quá hạn
4	Tiền phạt trả muộn	5.000 đồng / ngày trễ
5	Điều kiện mượn	Thẻ còn hạn, không có sách quá hạn, sách còn bản, không mượn trùng cuốn đang mượn
6	Xóa dữ liệu	Không xóa sách / độc giả đã có lịch sử mượn

2.3. Tác nhân của hệ thống

- **Quản trị viên (admin):** có toàn quyền, quản lý tài khoản thủ thư.
- **Thủ thư:** quản lý sách, độc giả, thực hiện mượn – trả – gia hạn và xem thống kê.
- **Quản trị hệ thống (DevOps):** vận hành hạ tầng qua pgAdmin, Grafana, Prometheus, Loki.

2.4. Yêu cầu chức năng

Bảng 2.3. *Yêu cầu chức năng của ứng dụng*

STT	Chức năng	Mô tả
1	Đăng nhập / đăng xuất	Thủ thư đăng nhập bằng tài khoản được cấp; khóa tạm thời khi nhập sai nhiều lần; đổi mật khẩu
2	Tổng quan	Hiển thị số đầu sách, bản sách, độc giả, phiếu đang mượn, phiếu quá hạn; biểu đồ mượn – trả 7 ngày; top sách mượn nhiều; cơ cấu kho theo thể loại

STT	Chức năng	Mô tả
3	Quản lý kho sách	Thêm, sửa, xóa sách; quản lý ISBN, thể loại, NXB, năm, vị trí kệ, số lượng; tìm kiếm theo tên/tác giả/ISBN, lọc theo thể loại; hiển thị số bản còn lại
4	Quản lý độc giả	Cấp thẻ, sửa, xóa độc giả; phân loại đối tượng; quản lý hạn thẻ; tìm kiếm; xem lịch sử mượn của từng độc giả
5	Mượn sách	Lập phiếu mượn, kiểm tra các điều kiện nghiệp vụ, tự tính hạn trả
6	Trả sách	Ghi nhận trả sách, tự động tính tiền phạt nếu trễ hạn
7	Gia hạn	Gia hạn phiếu mượn thêm 7 ngày theo quy định
8	Sách quá hạn	Danh sách phiếu quá hạn kèm số ngày trễ và phạt tạm tính, cảnh báo trên menu
9	Quản lý tài khoản	Quản trị viên tạo / xóa tài khoản thủ thư, phân quyền

2.5. Yêu cầu phi chức năng

- **Triển khai:** toàn bộ dịch vụ chạy bằng Docker Compose, khởi động bằng một lệnh, dữ liệu bền vững qua volume.
- **Bảo mật:** truy cập qua HTTPS; mật khẩu được băm; container chạy quyền tối thiểu; cơ sở dữ liệu không lộ ra ngoài.
- **Giám sát được:** theo dõi được tài nguyên container, lưu lượng web, tình trạng cơ sở dữ liệu và các thao tác nghiệp vụ.
- **Truy vết được:** log được thu thập tập trung, truy vấn được theo thời gian và nội dung.
- **Toàn vẹn dữ liệu:** các thao tác mượn – trả thực hiện trong giao dịch, không cho mượn vượt số bản kể cả khi nhiều thủ thư thao tác đồng thời.
- **Dễ sử dụng:** giao diện tiếng Việt, bố cục rõ ràng, có thông báo lỗi cụ thể, dùng được trên điện thoại.

2.6. Kế hoạch thực hiện

Bảng 2.4. *Các giai đoạn thực hiện và commit tương ứng*

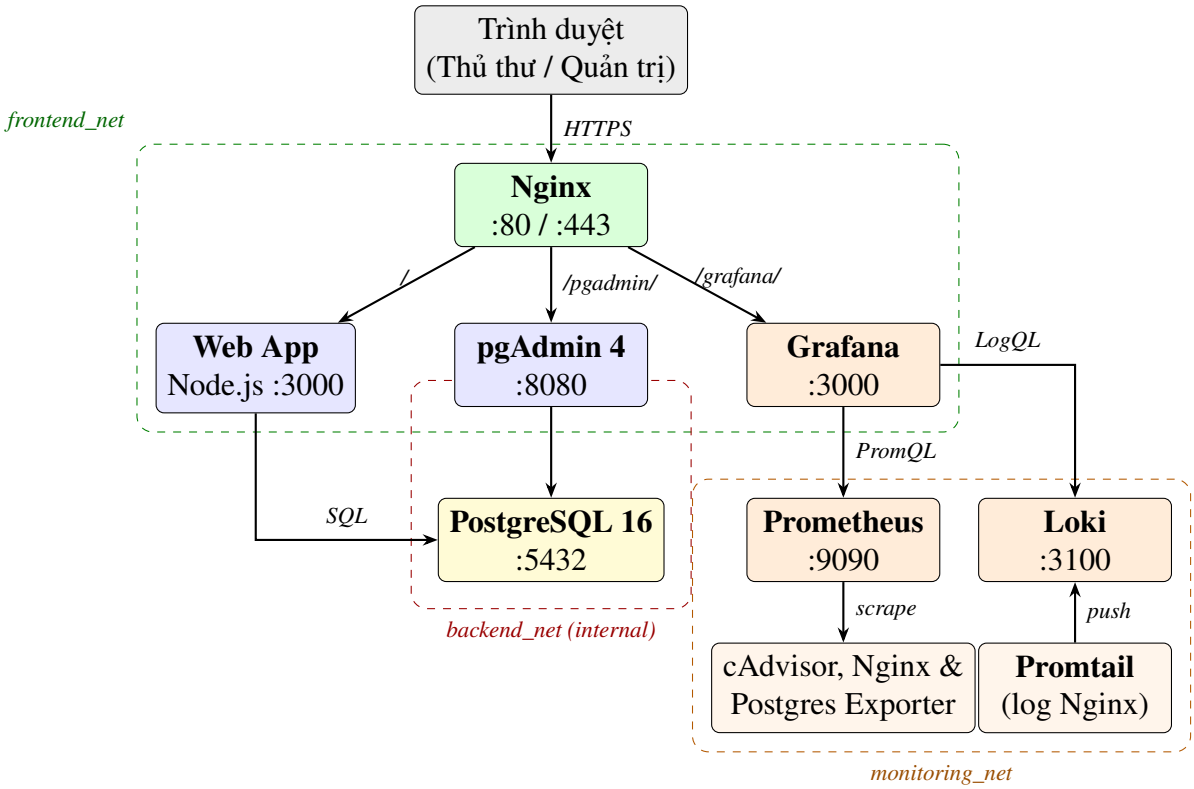
GĐ	Công việc	Kết quả
1	Xây dựng ứng dụng, PostgreSQL, pgAdmin, Nginx reverse proxy + SSL	Commit 1
2	Tích hợp Prometheus, Grafana và các exporter	Commit 2
3	Triển khai Loki, Promtail; hardening hệ thống	Commit 3
4	Rà soát, sửa lỗi cấu hình, nâng cấp chức năng ứng dụng, viết báo cáo	Các commit `fix`/`feat` bổ sung

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Chương này trình bày thiết kế kiến trúc tổng thể, thiết kế mạng, cơ sở dữ liệu, API của ứng dụng cùng thiết kế các thành phần giám sát, log và bảo mật.

3.1. Kiến trúc tổng thể

Hệ thống gồm 11 container chạy trên cùng một máy chủ Docker, được chia thành ba nhóm: nhóm ứng dụng (Nginx, Web App, pgAdmin), nhóm dữ liệu (PostgreSQL) và nhóm giám sát (Prometheus, Grafana, Loki, Promtail và ba exporter). Kiến trúc được minh họa trong Hình 3.1.



Hình 3.1. *Kiến trúc tổng thể của hệ thống*

Luồng hoạt động chính:

1. Người dùng truy cập `http://localhost`, Nginx trả mã 301 chuyển sang `https://localhost`.
2. Nginx kết thúc TLS, gắn security header và chuyển tiếp yêu cầu tới dịch vụ tương ứng theo đường dẫn.
3. Web App xử lý nghiệp vụ, truy vấn PostgreSQL qua mạng `backend_net`.
4. Prometheus định kỳ 5 giây thu thập metrics từ Web App và các exporter; Promtail đọc log Nginx và đẩy về Loki.
5. Quản trị viên xem dashboard và truy vấn log trên Grafana tại `https://localhos`

t/grafana/.

3.2. Thiết kế mạng

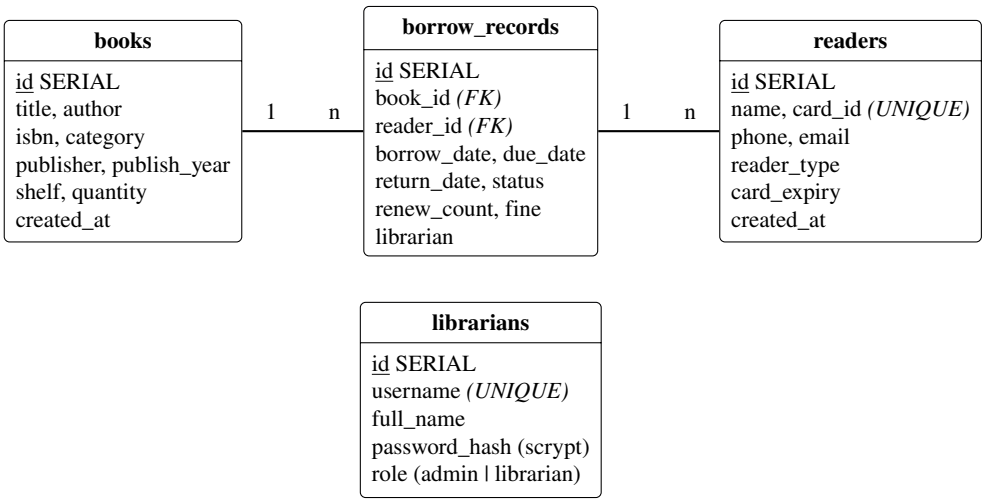
Bảng 3.1. *Phân vùng mạng Docker*

Mạng	Thành viên	Mục đích
frontend_net	Nginx, Web App, pgAdmin, Grafana	Nginx chuyển tiếp yêu cầu tới các dịch vụ có giao diện
backend_net (`internal: true`)	PostgreSQL, Web App, pgAdmin, Postgres Exporter	Truy cập cơ sở dữ liệu; không có đường ra Internet, không publish cổng
monitoring_net	Prometheus, Grafana, Loki, Promtail, các exporter, Web App, Nginx	Thu thập metrics và log

Chỉ Nginx publish cổng 80, 443 ra máy chủ. Prometheus chỉ bind `127.0.0.1:9090` để quản trị viên xem trên chính máy chủ. Trang `stub_status` của Nginx nằm ở cổng nội bộ 8080, không publish.

3.3. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu `library_system` gồm 4 bảng, quan hệ giữa các bảng được thể hiện trong Hình 3.2.



Hình 3.2. *Sơ đồ quan hệ thực thể (ERD)*

- **books:** mỗi dòng là một đầu sách; `quantity` là tổng số bản. Số bản còn lại không lưu cố định mà được tính bằng `quantity` trừ số phiếu đang mượn, tránh sai lệch dữ liệu.
- **readers:** thông tin độc giả, mã thẻ là duy nhất, có hạn thẻ `card_expiry`.
- **borrow_records:** mỗi dòng là một phiếu mượn một cuốn sách; `status` nhận giá trị `borrowing` hoặc `returned`; phiếu quá hạn là phiếu `borrowing` có `due_date` nhỏ hơn thời điểm hiện tại.

- **librarians:** tài khoản đăng nhập, mật khẩu được băm bằng thuật toán scrypt kèm salt ngẫu nhiên.

Lược đồ được ứng dụng tự khởi tạo và nâng cấp khi khởi động (`CREATE TABLE IF NOT EXISTS`, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`) nên dữ liệu cũ được giữ nguyên khi cập nhật phiên bản.

3.4. Thiết kế API

Ứng dụng cung cấp REST API định dạng JSON (Bảng 3.2). Mọi API nghiệp vụ yêu cầu cookie phiên đăng nhập.

Bảng 3.2. *Danh sách API của ứng dụng*

Method	**Đường dẫn**	**Chức năng**
POST	/api/login, /api/logout	Đăng nhập, đăng xuất
GET	/api/me	Thông tin người đang đăng nhập
POST	/api/me/password	Đổi mật khẩu
GET	/api/stats	Số liệu thống kê cho trang tổng quan
GET, POST	/api/books	Danh sách (tìm kiếm, lọc), thêm sách
PUT, DELETE	/api/books/:id	Sửa, xóa sách
GET	/api/categories	Danh sách thể loại
GET, POST	/api/readers	Danh sách (tìm kiếm), cấp thẻ độc giả
PUT, DELETE	/api/readers/:id	Sửa, xóa độc giả
GET	/api/readers/:id/history	Lịch sử mượn của độc giả
GET, POST	/api/borrow	Danh sách phiếu (lọc theo trạng thái), lập phiếu mượn
POST	/api/borrow/:id/return	Trả sách, tính tiền phạt
POST	/api/borrow/:id/renew	Gia hạn phiếu mượn
GET, POST, DELETE	/api/users, /api/users/:id	Quản lý tài khoản thủ thư (chỉ admin)
GET	/metrics, /healthz	Metrics cho Prometheus, kiểm tra sức khỏe (Nginx chặn /metrics từ bên ngoài)

3.5. Thiết kế xử lý mượn sách

Nghiệp vụ mượn sách cần đảm bảo không cho mượn vượt số bản kể cả khi hai thủ thư cùng thao tác. Vì vậy toàn bộ việc kiểm tra và ghi phiếu được thực hiện trong một

giao dịch, dòng sách và dòng độc giả được khóa bằng `SELECT ... FOR UPDATE` (Thuật toán 1).

Thuật toán 1 Lập phiếu mượn sách

Đầu vào: Mã sách b, mã độc giả r, số ngày mượn d
Đầu ra: Phiếu mượn mới hoặc thông báo lỗi

BEGIN giao dịch
Khóa dòng sách b; **if** không tồn tại **then** báo lỗi 404
if số phiếu đang mượn của $b \geq$ quantity **then**
    ROLLBACK; báo lỗi "Sách đã được mượn hết"
end if
Khóa dòng độc giả r, đếm số sách đang mượn n và số sách quá hạn q
if thẻ hết hạn **or** $q > 0$ **or** $n \geq 5$ **or** r đang mượn b **then**
    ROLLBACK; báo lỗi tương ứng
end if
Thêm phiếu: due_date $\leftarrow$ hiện tại $+d$ ngày
COMMIT; tăng metric `library_loan_operations_total{operation="borrow"}`
return phiếu mượn

Khi trả sách, tiền phạt được tính theo Công thức 3.1:

$$\text{Phạt} = \max\left(0, \left\lceil \frac{t_{\text{trả}} - t_{\text{hạn}}}{1 \text{ ngày}} \right\rceil\right) \times 5000 \text{ (đồng)} \tag{3.1}$$

3.6. Thiết kế giám sát và log

Bảng 3.3. *Các target được Prometheus thu thập*

Job	Target	Số liệu chính
library-app	app:3000/metrics	Request theo route và mã trạng thái, số lần đăng nhập, số thao tác mượn/trả/gia hạn, CPU/heap Node.js
cadvisor	cadvisor:8080	CPU, RAM, mạng của từng container
postgres-exporter	postgres-exporter:9187	pg_up, số kết nối, commit/rollback
nginx-exporter	nginx-exporter:9113	nginx_up, kết nối đang mở, tổng request

Dashboard Grafana *Library System Overview* được provision từ file JSON trong repository, gồm 13 panel: trạng thái PostgreSQL/Nginx, số kết nối, CPU và RAM từng container, request của ứng dụng và Nginx, commit/rollback của cơ sở dữ liệu, số log lỗi theo phút (truy vấn Loki), số thao tác nghiệp vụ, kết quả đăng nhập và luồng access log.

Nginx ghi access log dạng JSON (`time_local`, `remote_addr`, `request`, `status`,

body_bytes_sent, request_time, http_referrer) ra thư mục logs/nginx. Promtail đọc thư mục này với nhãn job="nginx" và đẩy về Loki; nhờ định dạng JSON, LogQL có thể dùng bộ phân tích | json để lọc theo từng trường.

3.7. Thiết kế bảo mật

Bảng 3.4. *Các biện pháp hardening*

STT	Biện pháp	Thực hiện
1	Non-root container	Web App chạy USER node (uid 1000); pgAdmin chạy user pgadmin (uid 5050) ở cổng 8080
2	Hạn chế quyền container	no-new-privileges cho mọi container; Web App read_only: true và cap_drop: ALL
3	Cô lập mạng	3 mạng riêng; backend_net là mạng internal; PostgreSQL không publish cổng
4	Mật khẩu mạnh, quản lý bí mật	Mật khẩu $\geq$ 12 ký tự trong file .env (không commit); compose báo lỗi nếu thiếu biến
5	HTTPS	Chứng chỉ tự ký, TLS 1.2/1.3, chuyển hướng HTTP $\rightarrow$ HTTPS; private key không đưa lên Git
6	Security header	X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy, Strict-Transport-Security; ẩn phiên bản Nginx (server_tokens off) và header X-Powered-By
7	Giới hạn truy cập endpoint nội bộ	Chặn /metrics từ ngoài; stub_status chỉ cho dải IP Docker; Prometheus bind 127.0.0.1
8	Xác thực ứng dụng	Bắt buộc đăng nhập; mật khẩu băm scrypt; cookie HttpOnly, Secure, SameSite=Strict; phân quyền admin/thủ thư
9	Chống dò mật khẩu	Nginx limit_req 5 request/phút cho /api/login; ứng dụng khóa 15 phút sau 5 lần sai
10	An toàn mã nguồn	Truy vấn tham số hóa (chống SQL injection), escape HTML (chống XSS), kiểm tra dữ liệu đầu vào, giới hạn kích thước request

CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ

Chương này trình bày môi trường, các bước triển khai và kết quả vận hành thực tế của hệ thống, kèm hình ảnh minh chứng cho từng yêu cầu của đề bài.

4.1. Môi trường triển khai

Bảng 4.1. *Môi trường và phiên bản phần mềm*

Thành phần	**Phiên bản / image**
Hệ điều hành máy chủ	Ubuntu Linux, Docker Engine + Docker Compose plugin
Web App	`node:20-alpine`, Express 4, pg 8, prom-client 15
Cơ sở dữ liệu	`postgres:16-alpine`, `dpage/pgadmin4`
Reverse proxy	`nginx:alpine`
Giám sát	`prom/prometheus`, `grafana/grafana`, `gcr.io/cadvisor/cadvisor`, `prometheuscommunity/postgres-exporter`, `nginx/nginx-prometheus-exporter`
Log tập trung	`grafana/loki`, `grafana/promtail`

4.2. Cấu trúc mã nguồn

Mã nguồn được lưu trữ tại `https://github.com/huynhpham13790-rgb/DTC245200357-LibraryManagement` với cấu trúc:

```
library-management/
|-- docker-compose.yml     # 11 dich vu, 3 mang, 3 volume
|-- .env.example           # mau bien moi truong
|-- README.md              # huong dan chay
|-- app/                   # Web App Node.js
|   |-- Dockerfile         # USER node (non-root)
|   |-- server.js          # REST API, xac thuc, metrics
|   `-- public/            # giao dien HTML/JS/CSS
|-- nginx/nginx.conf       # proxy, SSL, header, rate limit
|-- monitoring/
|   |-- prometheus/prometheus.yml
|   |-- promtail/promtail-config.yml
|   `-- grafana/           # datasource + dashboard
`-- bao_cao/               # bao cao LaTeX nay
```

4.3. Các bước triển khai

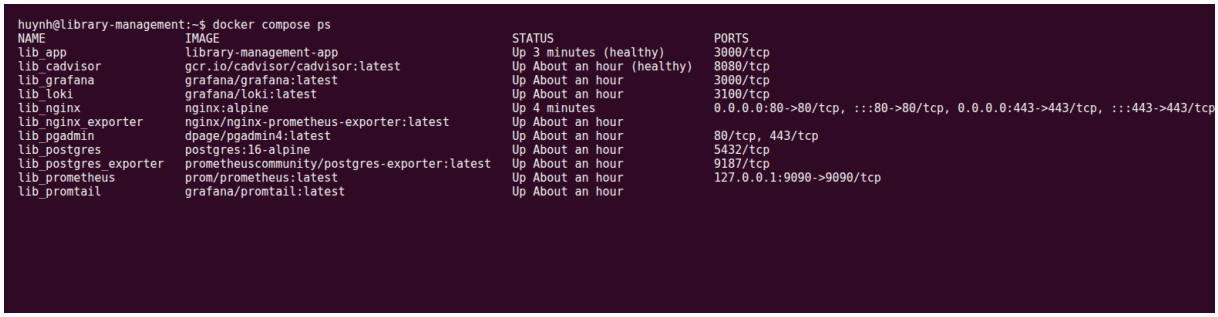
4.3.1. Tạo chứng chỉ SSL và biến môi trường

```
mkdir -p nginx/ssl
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout nginx/ssl/key.pem -out nginx/ssl/cert.pem \
  -subj "/C=VN/ST=ThaiNguyen/L=ThaiNguyen/O=ICTU/CN=library.local"
cp .env.example .env     # sau do doi toan bo mat khau
```

4.3.2. Khởi chạy hệ thống

```
docker compose up -d --build
docker compose ps
```

Kết quả 11 container đều ở trạng thái *Up* được thể hiện trong Hình 4.1. Chỉ Nginx publish cổng 80/443 ra ngoài; Prometheus chỉ bind `127.0.0.1`.



Hình 4.1. *Kết quả lệnh docker compose ps*

4.3.3. Cấu hình Nginx chính

Đoạn cấu hình quan trọng của Nginx (rút gọn):

```
server { listen 80; return 301 https://$host$request_uri; }
server {
    listen 443 ssl;
    ssl_protocols TLSv1.2 TLSv1.3;
    location /           { proxy_pass http://app:3000; }
    location /pgadmin/ {
        proxy_pass http://pgadmin:8080/;
        proxy_set_header X-Script-Name /pgadmin; }
    location /grafana/   { proxy_pass http://grafana:3000; }
    location = /api/login { limit_req zone=login burst=5 nodelay;
                            proxy_pass http://app:3000; }
    location = /metrics  { return 404; }
}
```

4.4. Kết quả ứng dụng quản lý thư viện

4.4.1. Đăng nhập

Khi truy cập `https://localhost/`, người dùng được yêu cầu đăng nhập (Hình 4.2). Nếu nhập sai, hệ thống hiển thị thông báo lỗi; sau 5 lần sai liên tiếp, tài khoản tại địa chỉ IP đó bị khóa 15 phút.



Hình 4.2. *Màn hình đăng nhập (trái) và thông báo khi nhập sai mật khẩu (phải)*

4.4.2. Trang tổng quan

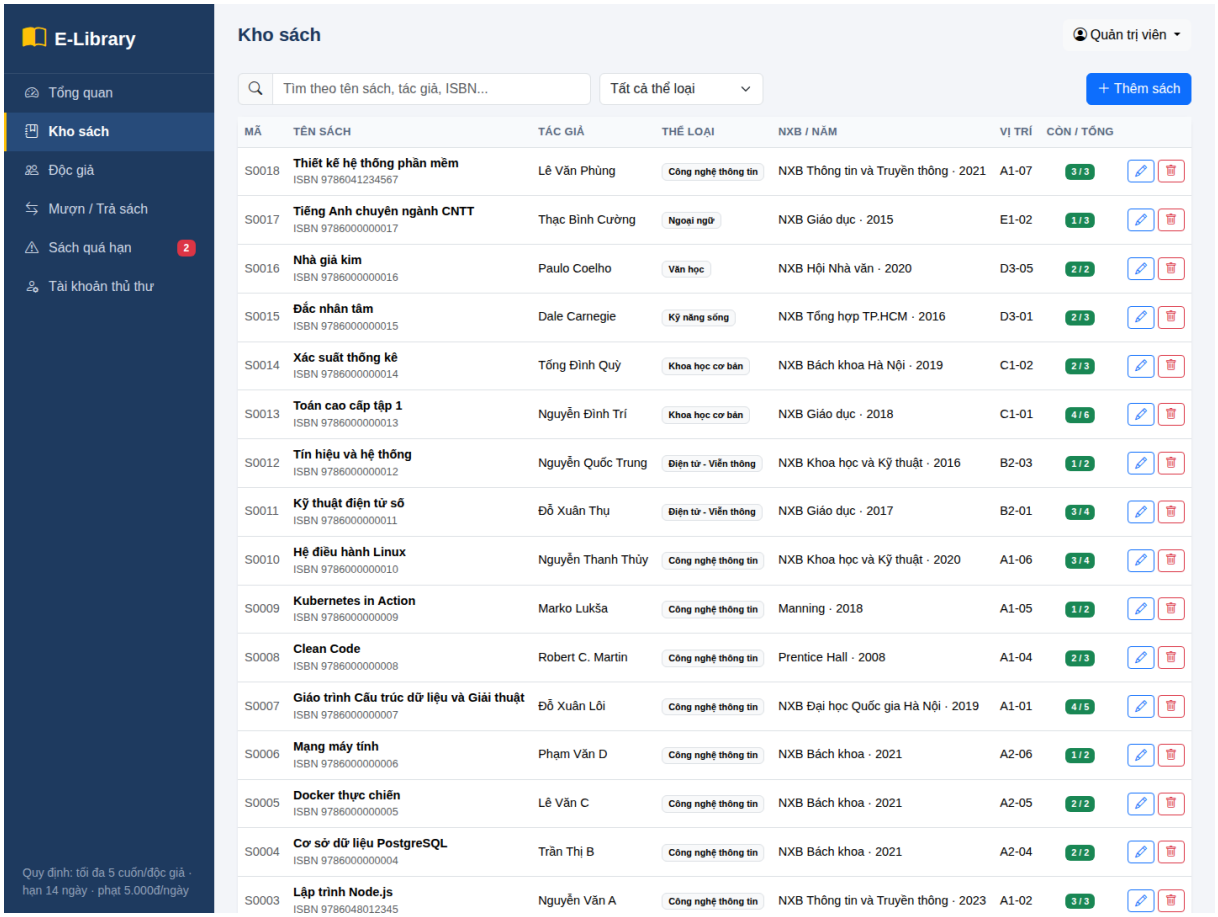
Sau khi đăng nhập, trang tổng quan (Hình 4.3) hiển thị số đầu sách, số bản sách, số độc giả, số phiếu đang mượn và quá hạn; biểu đồ lượt mượn – trả 7 ngày gần nhất; 5 cuốn được mượn nhiều nhất; cơ cấu kho theo thể loại và tổng tiền phạt đã thu. Menu bên trái hiển thị số phiếu quá hạn để thủ thư chú ý.



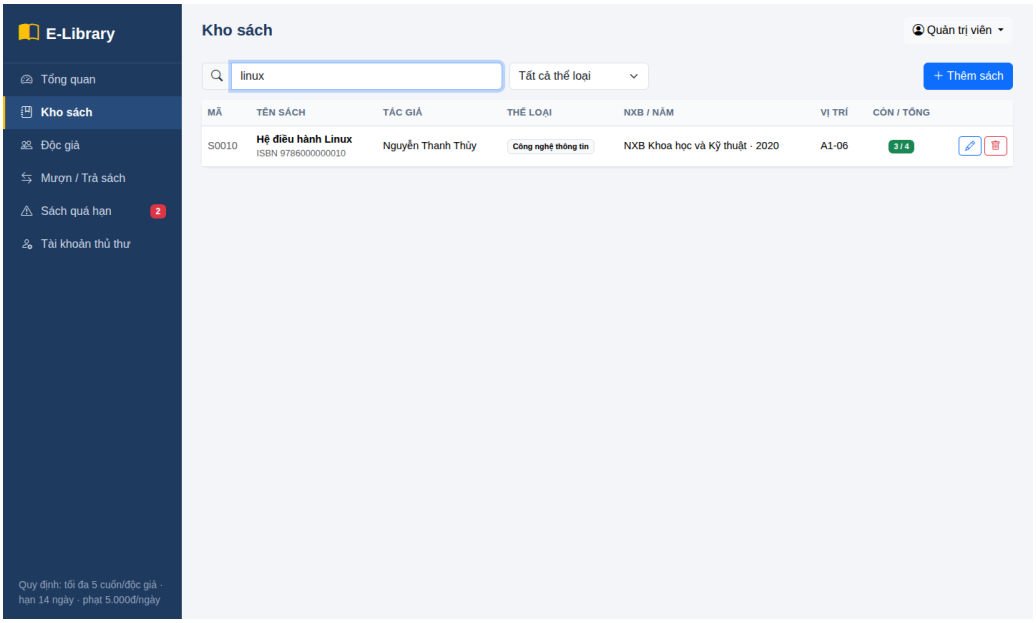
Hình 4.3. *Trang tổng quan của ứng dụng*

4.4.3. Quản lý kho sách

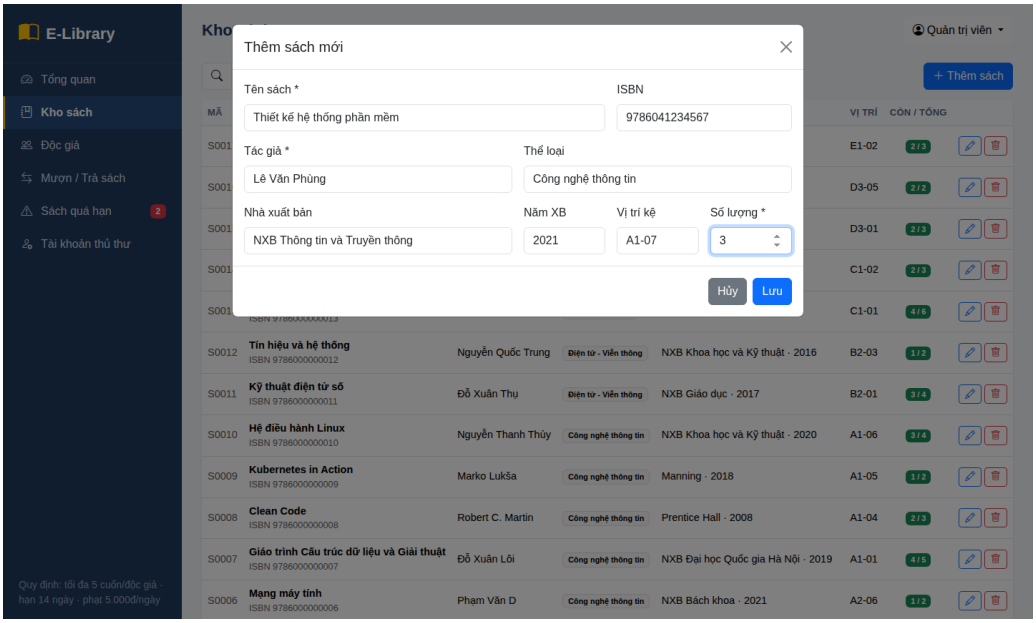
Trang Kho sách (Hình 4.4) liệt kê các đầu sách với mã, ISBN, tác giả, thể loại, nhà xuất bản, năm, vị trí kệ và số bản còn lại trên tổng số bản. Thủ thư có thể tìm kiếm theo tên, tác giả, ISBN, lọc theo thể loại (Hình 4.5), thêm hoặc sửa sách qua hộp thoại (Hình 4.6) và xóa sách chưa phát sinh lượt mượn.



Hình 4.4. *Danh sách kho sách*



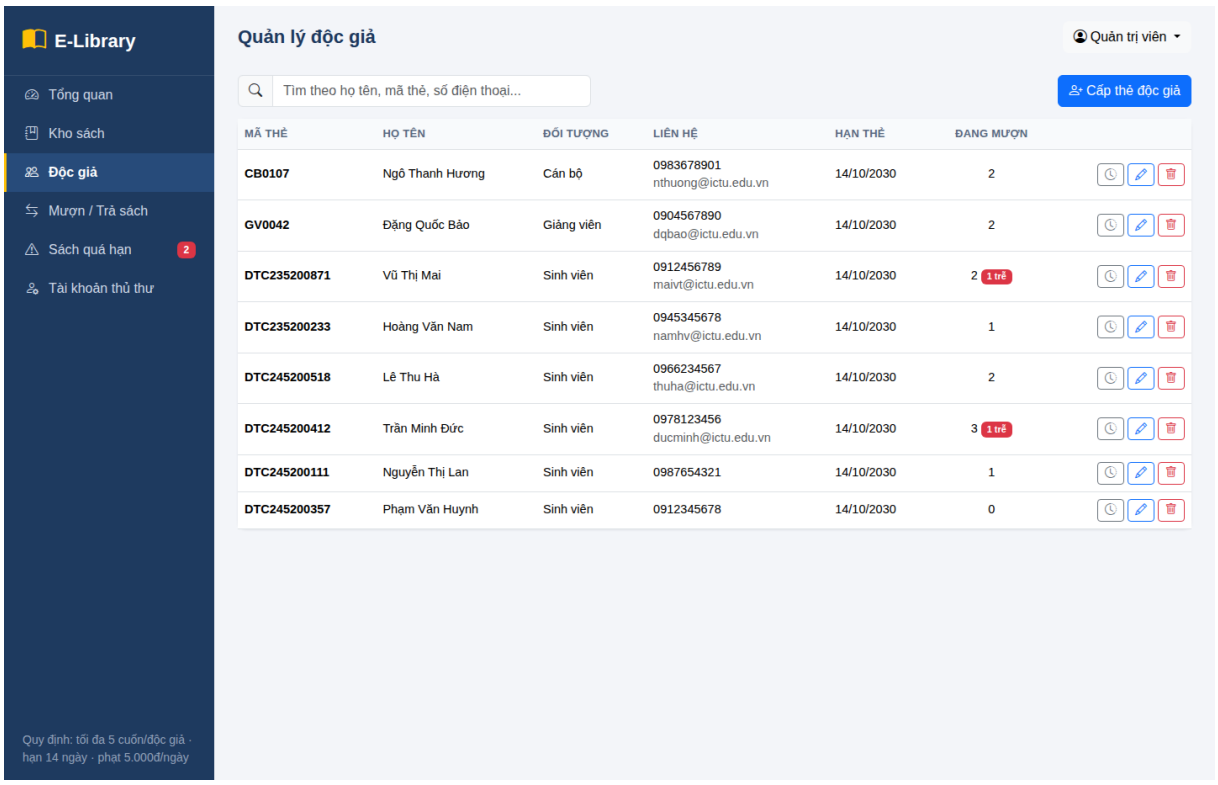
Hình 4.5. *Tìm kiếm sách theo từ khóa "linux"*



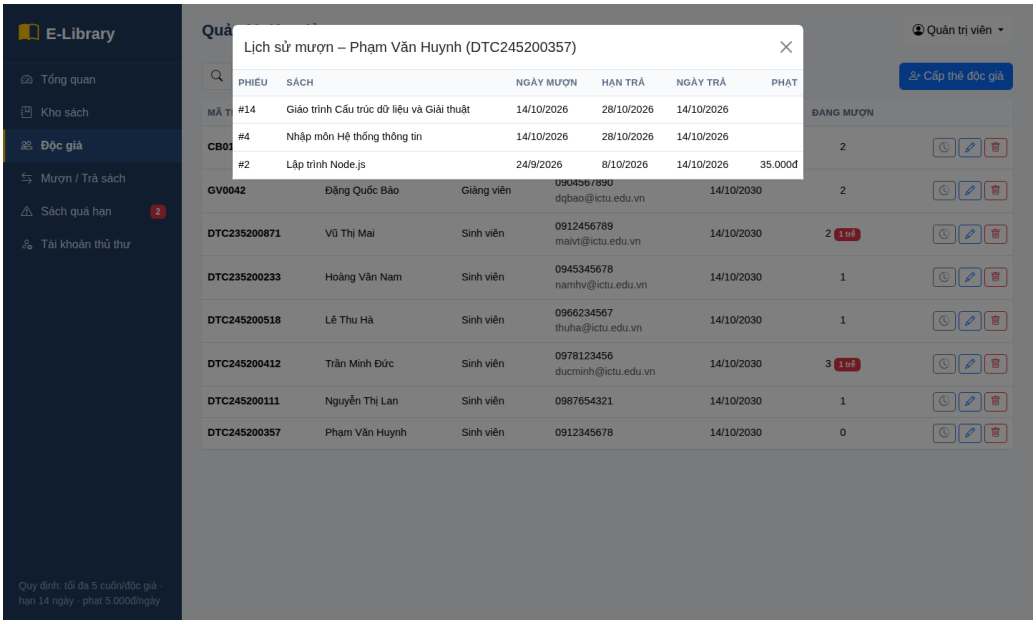
Hình 4.6. *Hộp thoại thêm sách mới*

4.4.4. Quản lý độc giả

Trang Độc giả (Hình 4.7) quản lý mã thẻ, đối tượng, thông tin liên hệ, hạn thẻ và số sách đang mượn của từng độc giả (đánh dấu đỏ nếu có sách trễ hạn). Nút lịch sử cho phép xem toàn bộ các lần mượn của một độc giả (Hình 4.8).



Hình 4.7. *Danh sách độc giả*

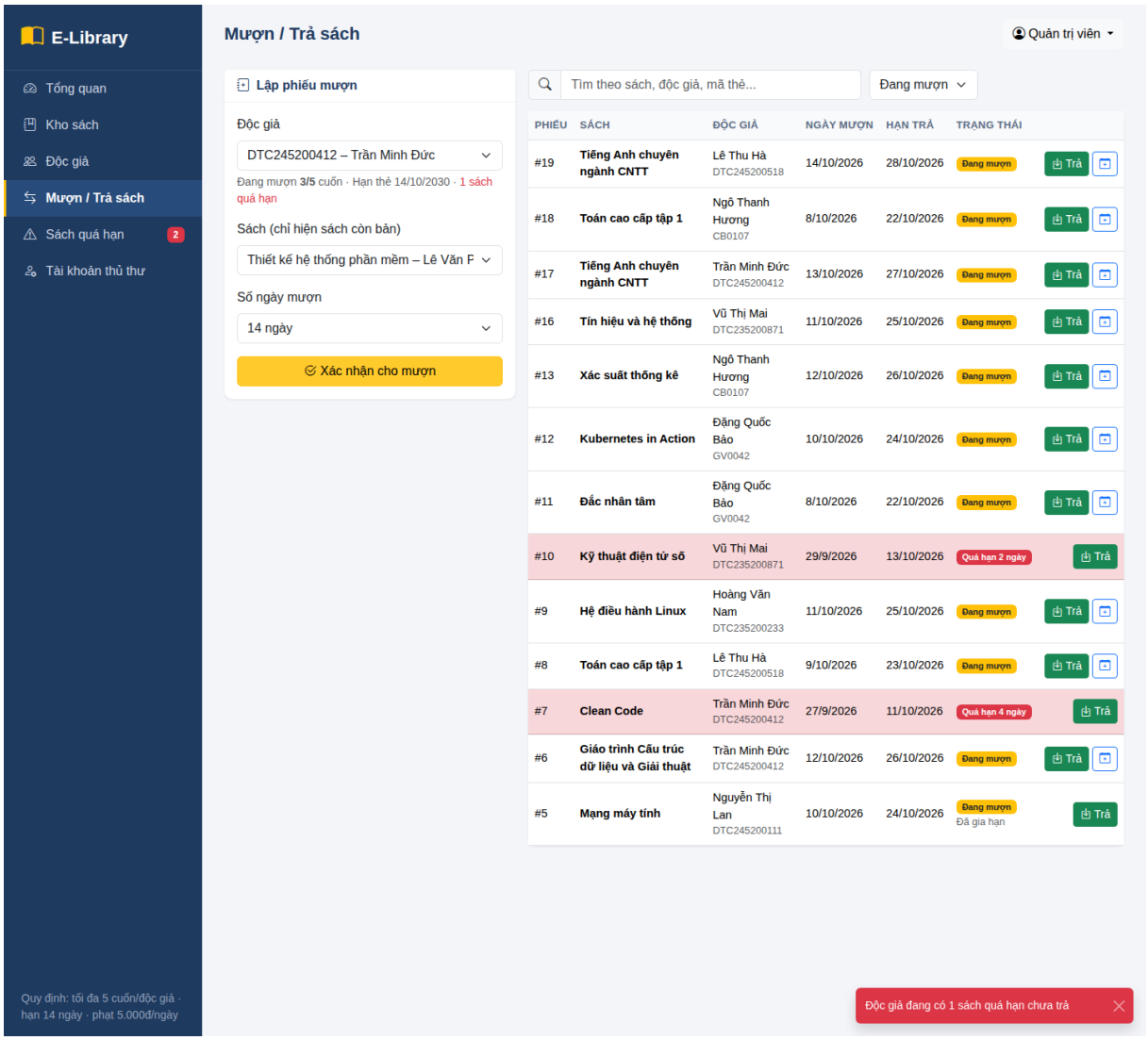


Hình 4.8. *Lịch sử mượn sách của một độc giả*

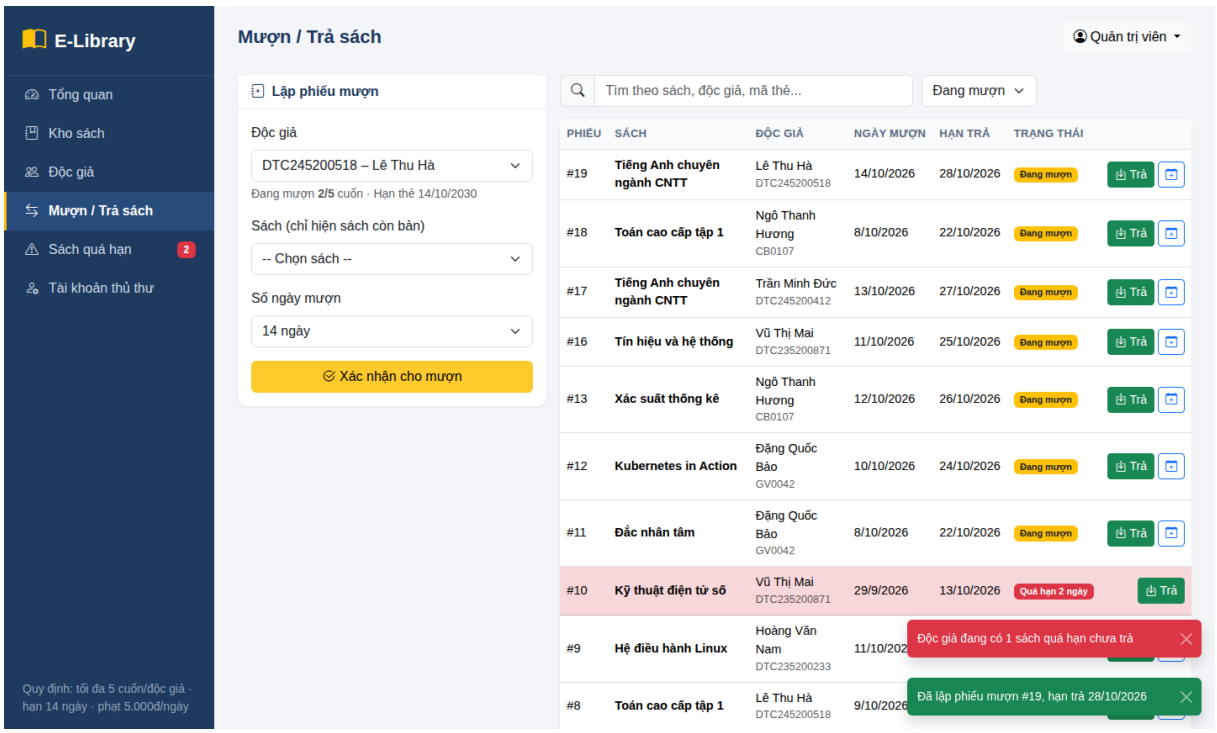
4.4.5. Mượn, trả và gia hạn sách

Trang Mượn / Trả sách gồm biểu mẫu lập phiếu và danh sách phiếu. Khi chọn độc giả, hệ thống hiển thị ngay số sách đang mượn, hạn thẻ và cảnh báo sách quá hạn. Hình 4.9 cho thấy độc giả Trần Minh Đức đang có một cuốn quá hạn nên yêu cầu mượn mới bị từ chối với thông báo cụ thể. Ngược lại, một độc giả hợp lệ được lập phiếu thành công và hệ thống báo hạn trả (Hình 4.10). Các phiếu quá hạn được tô đỏ, phiếu còn hạn

có thêm nút gia hạn.

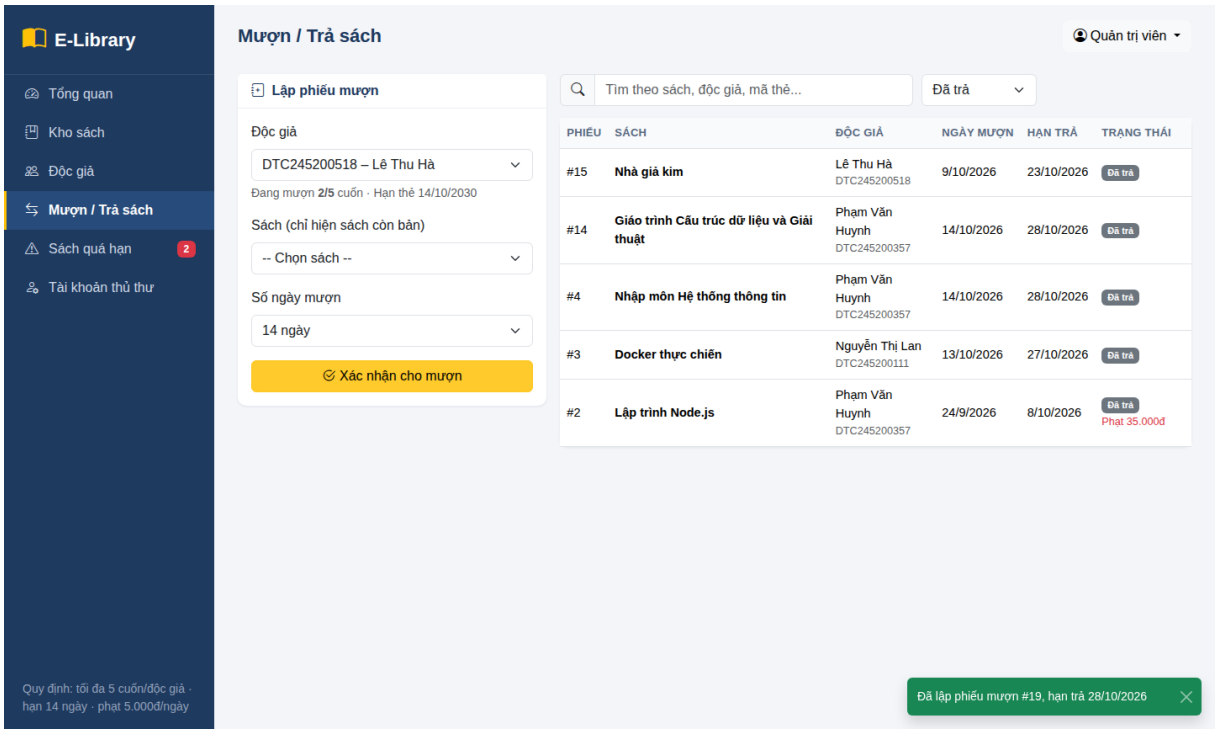


Hình 4.9. *Từ chối cho mượn khi độc giả đang có sách quá hạn*



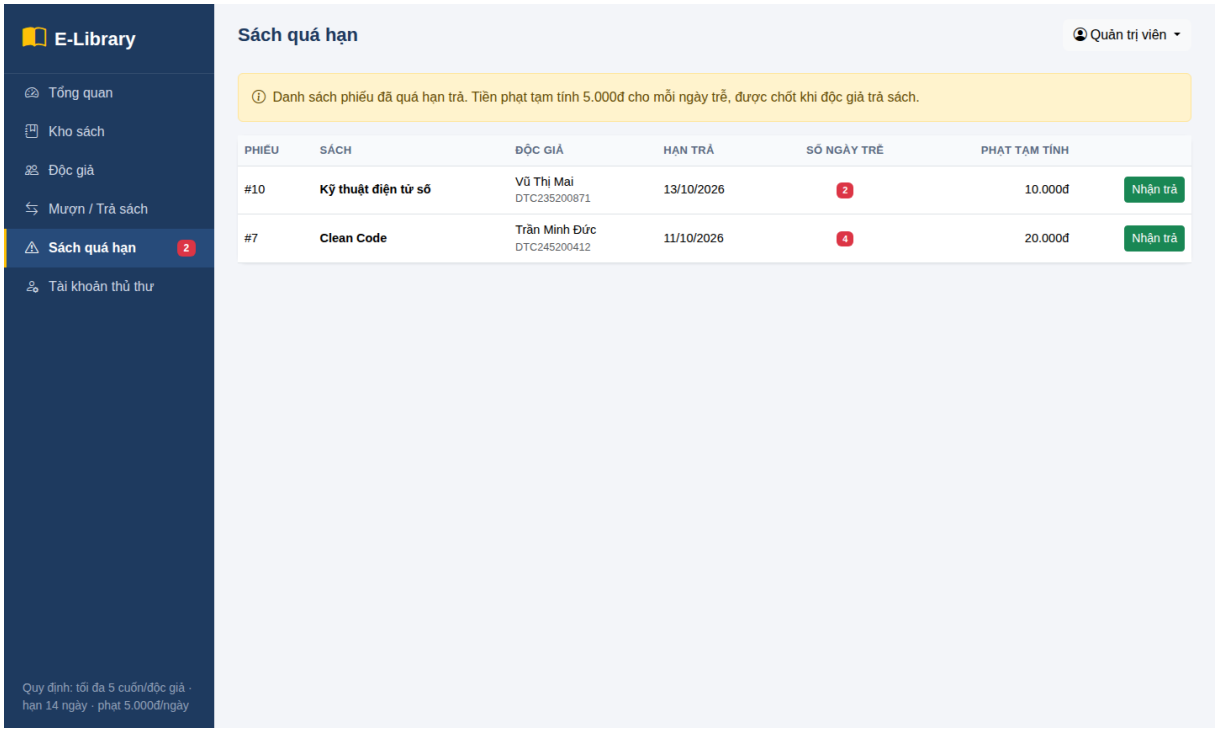
Hình 4.10. *Lập phiếu mượn thành công (thông báo góc phải dưới)*

Khi trả sách trễ hạn, tiền phạt được tự động tính và lưu vào phiếu (Hình 4.11).

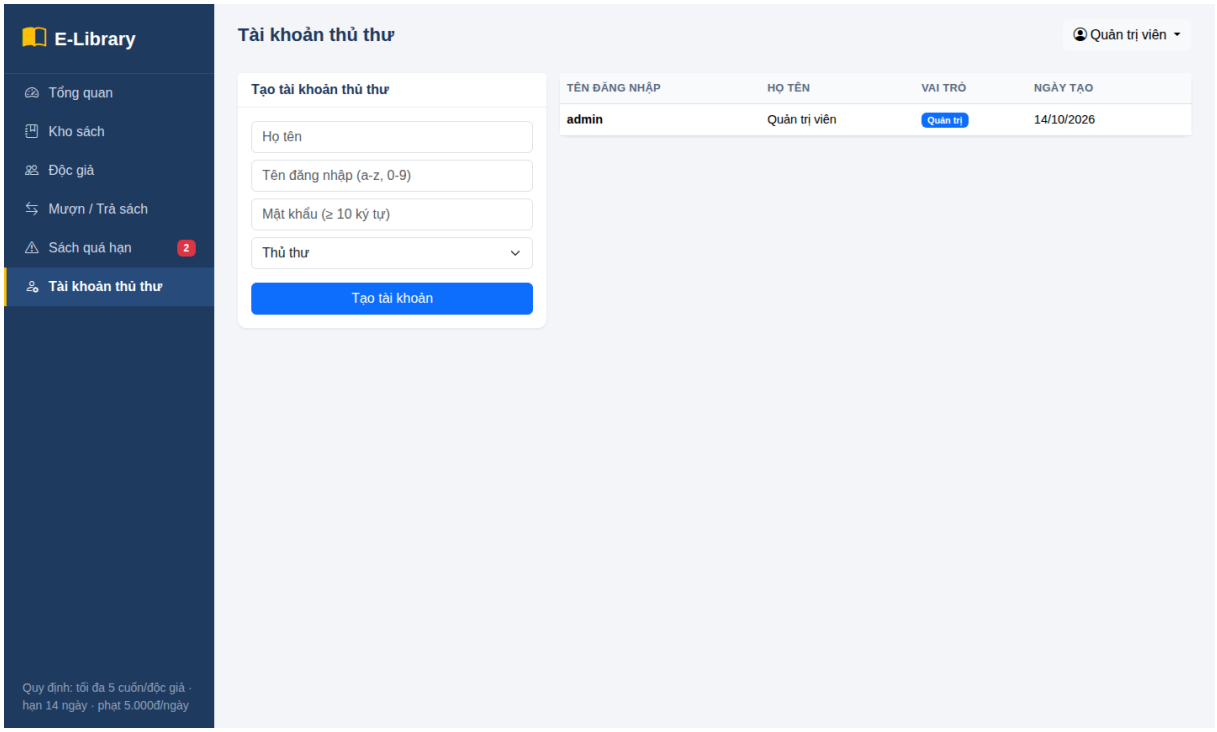


Hình 4.11. *Danh sách phiếu đã trả, có phiếu bị phạt do trễ hạn*

4.4.6. *Sách quá hạn và tài khoản thủ thư*

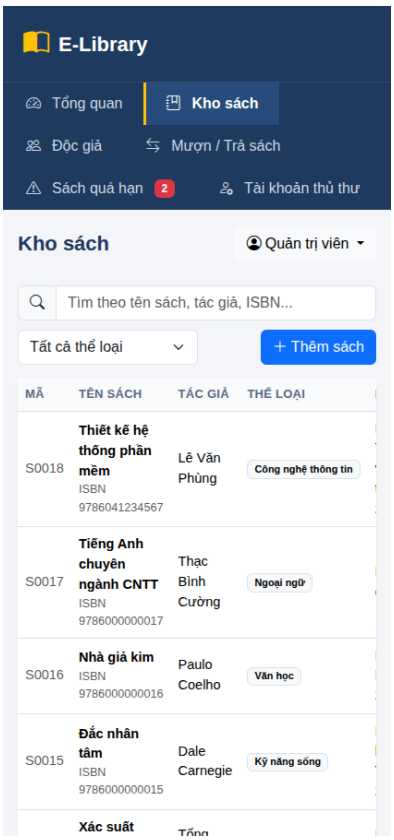


Hình 4.12. *Danh sách sách quá hạn kèm số ngày trễ và tiền phạt tạm tính*



Hình 4.13. *Quản lý tài khoản thủ thư (chỉ quản trị viên)*

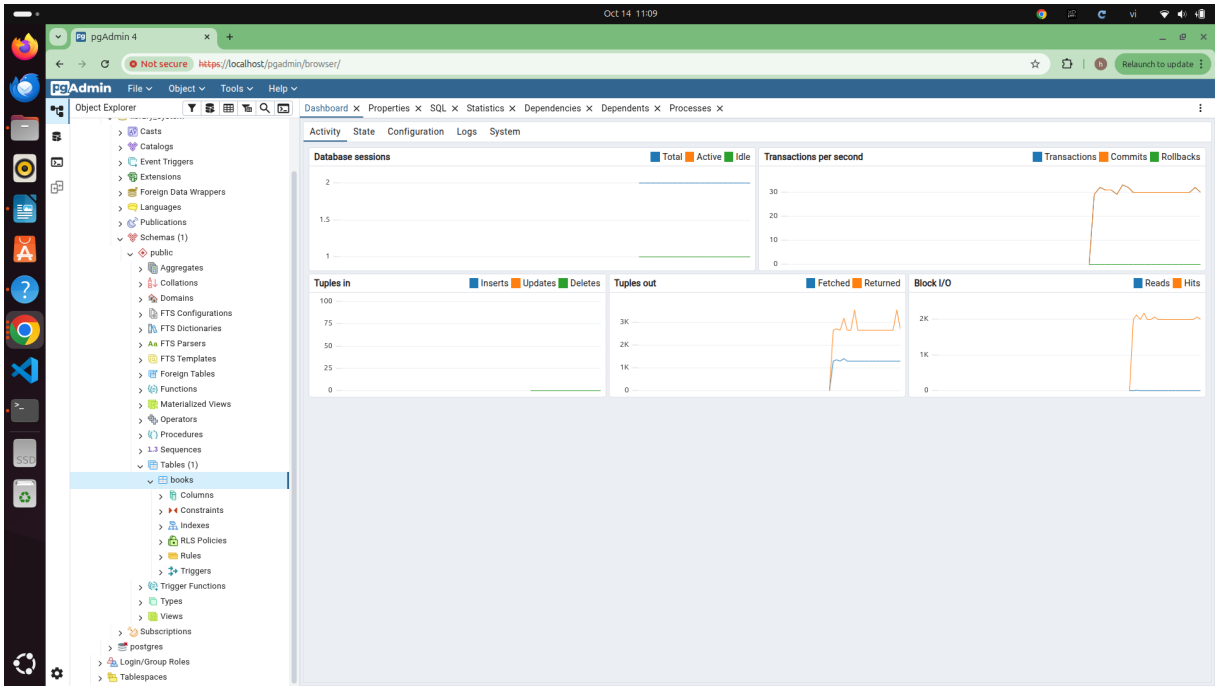
Giao diện được thiết kế co giãn, sử dụng được trên điện thoại (Hình 4.14).



Hình 4.14. *Giao diện trên màn hình điện thoại*

4.5. Kết quả cơ sở dữ liệu và pgAdmin

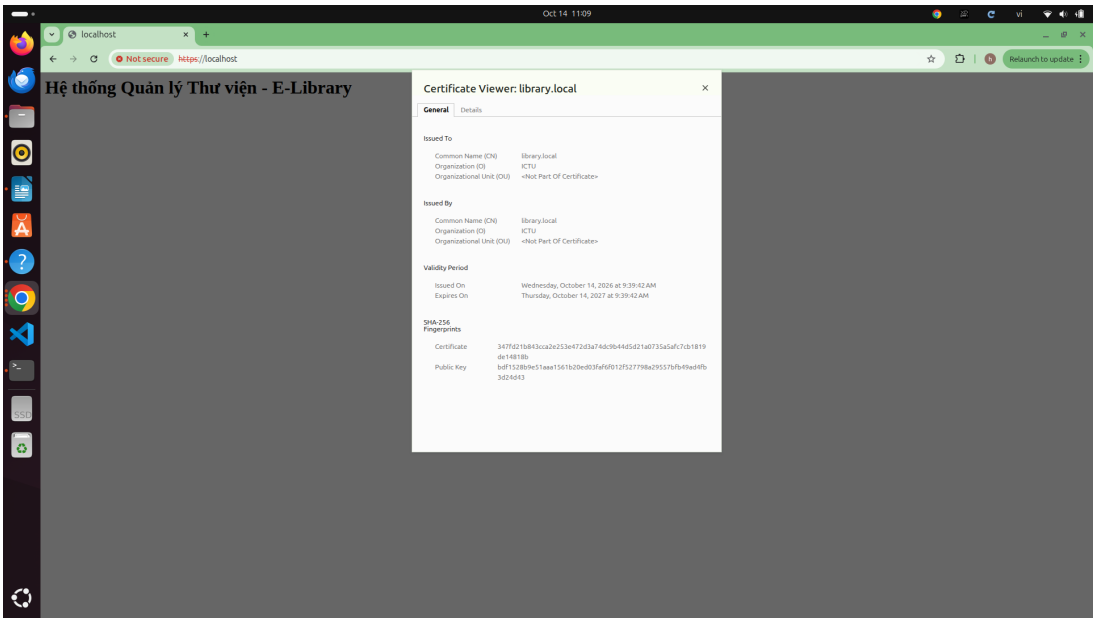
pgAdmin được truy cập tại `https://localhost/pgadmin/` thông qua Nginx. Sau khi đăng ký server với host `postgres`, cổng 5432, có thể duyệt các bảng của cơ sở dữ liệu `library_system` và theo dõi hoạt động (Hình 4.15).



Hình 4.15. *pgAdmin 4 kết nối tới PostgreSQL qua reverse proxy*

4.6. Kết quả Nginx reverse proxy và HTTPS

Trình duyệt kết nối qua HTTPS với chứng chỉ tự ký có Common Name `library.local`, tổ chức ICTU (Hình 4.16).



Hình 4.16. *Thông tin chứng chỉ SSL tự ký*

Hình 4.17 là kết quả kiểm tra thực tế bằng dòng lệnh: phản hồi chứa đủ các security header và HSTS, không lộ phiên bản Nginx hay header `X-Powered-By`; `/metrics` bị chặn (404); API trả 401 khi chưa đăng nhập; container ứng dụng chạy uid 1000 (node), pgAdmin chạy uid 5050; mạng `backend_net` là mạng internal; gửi liên tục yêu cầu đăng nhập bị Nginx trả 429.

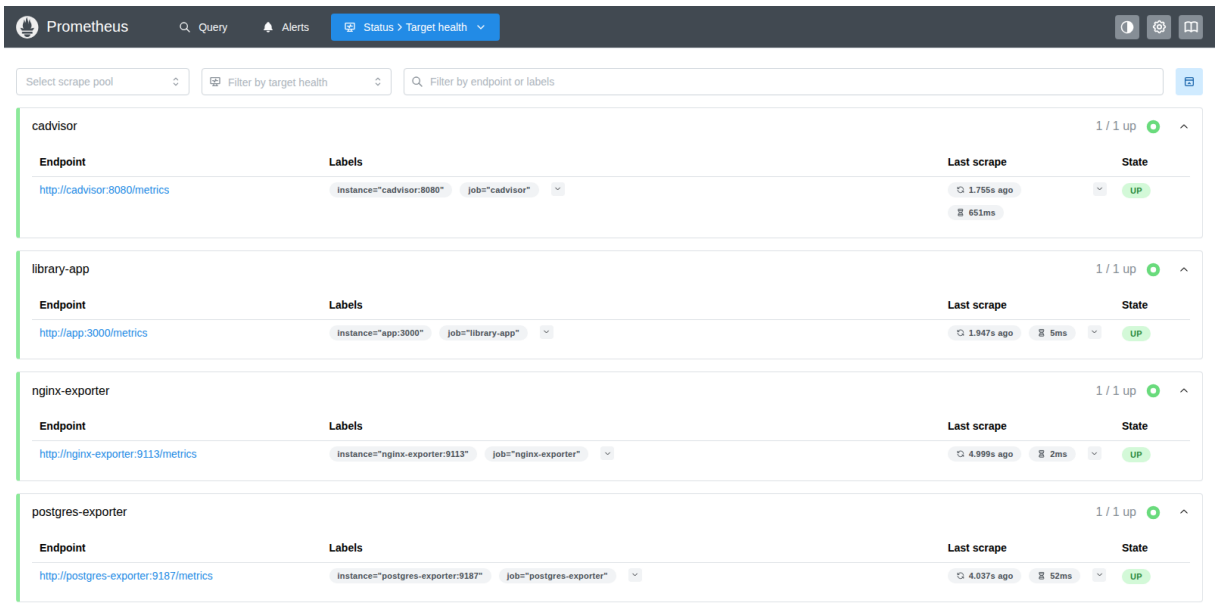
```
huynh@library-management:~$ curl -kI https://localhost/
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 14 Oct 2026 07:02:38 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 15650
Connection: keep-alive
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Strict-Transport-Security: max-age=31536000

huynh@library-management:~$ curl -sk -o /dev/null -w '%{http_code}\n' https://localhost/metrics
404
huynh@library-management:~$ curl -sk https://localhost/api/books
{"error":"Phiên đăng nhập đã hết hạn, vui lòng đăng nhập lại"}
huynh@library-management:~$ docker exec lib_app id
uid=1000(node) gid=1000(node) groups=1000(node),1000(node)
huynh@library-management:~$ docker exec lib_pgadmin id
uid=5050(pgadmin) gid=0(root) groups=0(root)
huynh@library-management:~$ docker network inspect library-management_backend_net -f '{{.Internal}}'
true
huynh@library-management:~$ for i in $(seq 1 8); do curl ... -X POST https://localhost/api/login; done
429 429 429 429 429 429 429 429
```

Hình 4.17. *Kiểm tra security header và các biện pháp hardening*

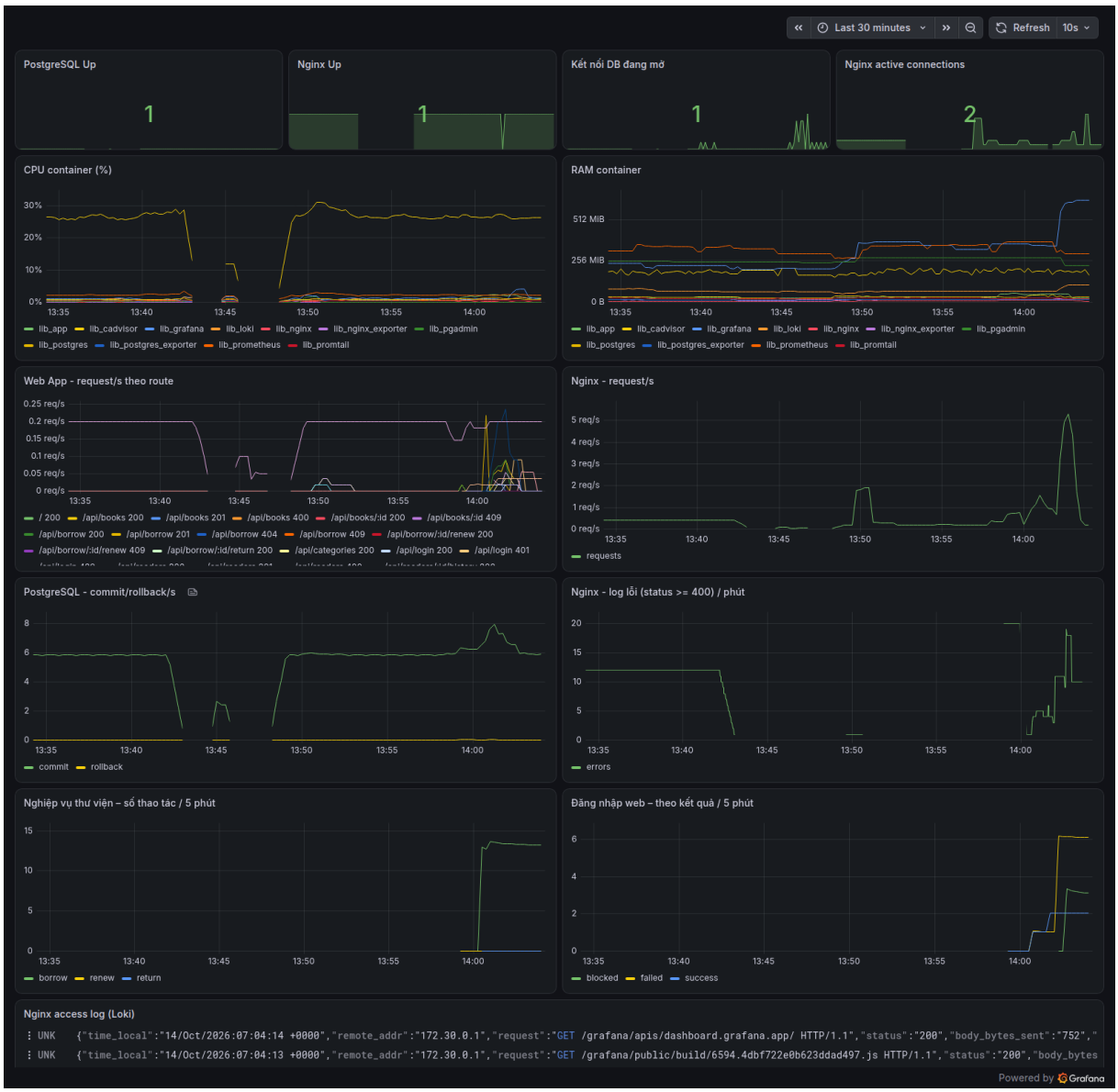
4.7. Kết quả giám sát với Prometheus và Grafana

Cả 4 target đều ở trạng thái UP (Hình 4.18).



Hình 4.18. *Các target trên Prometheus đều ở trạng thái UP*

Dashboard *Library System Overview* (Hình 4.19) giám sát đồng thời: **container** (CPU, RAM từng container `lib_*` qua cAdvisor), **web server** (trạng thái, kết nối, request/s của Nginx; request/s theo route và mã trạng thái của ứng dụng), **cơ sở dữ liệu** (pg_up, số kết nối, commit/rollback) và **nghiệp vụ** (số lượt mượn/trả/gia hạn, đăng nhập thành công/thất bại/bị chặn).



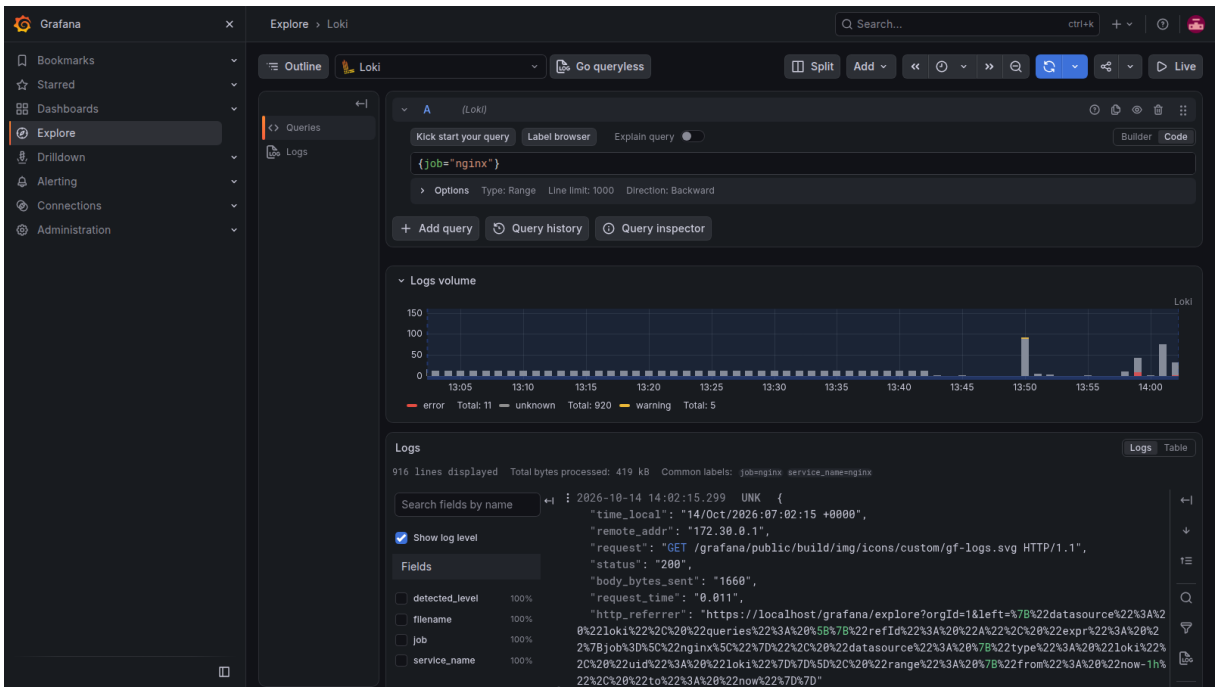
Hình 4.19. *Dashboard Grafana giám sát container, web server và cơ sở dữ liệu*

4.8. Kết quả log tập trung với Loki và LogQL

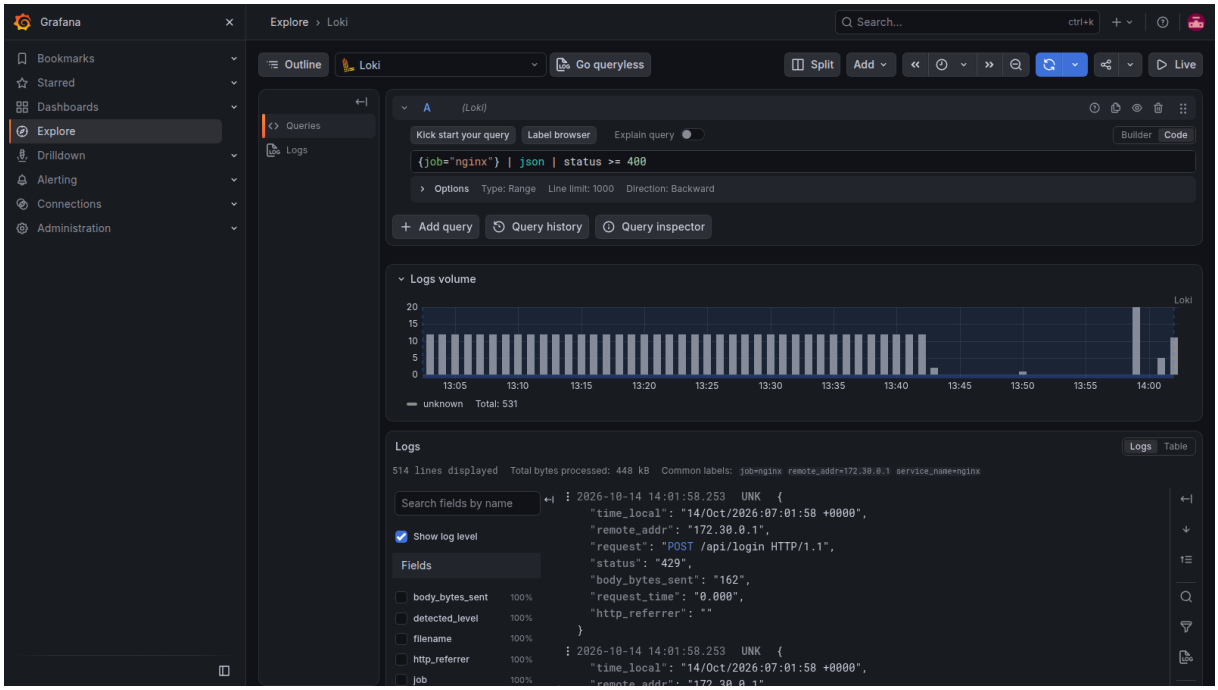
Log của Nginx được truy vấn trên Grafana Explore với datasource Loki. Bốn truy vấn LogQL được sử dụng:

Bảng 4.2. *Các truy vấn LogQL*

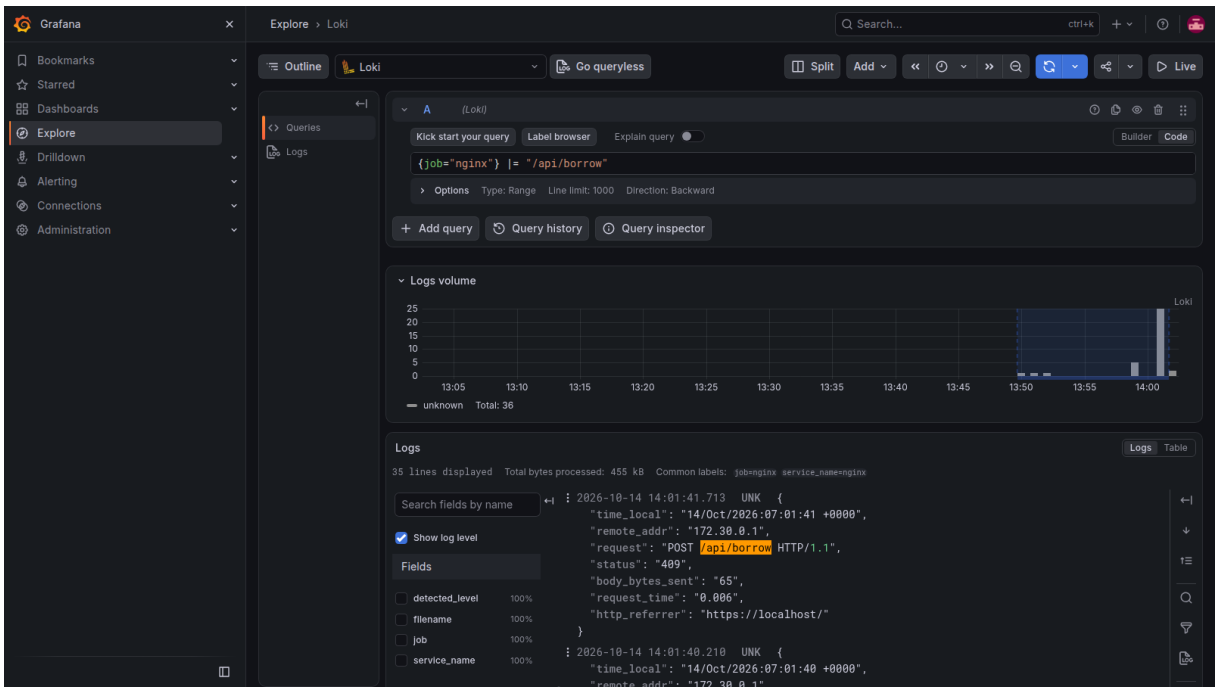
STT	Truy vấn	Ý nghĩa
1	`{job="nginx"}`	Toàn bộ log Nginx
2	`{job="nginx"} | json | status >= 400`	Các request lỗi 4xx, 5xx
3	`{job="nginx"} |= "/api/borrow"`	Log nghiệp vụ mượn sách
4	`{job="nginx"} |= "/api/login" | json | status =~ "401|429"`	Đăng nhập sai hoặc bị chặn – phát hiện dò mật khẩu



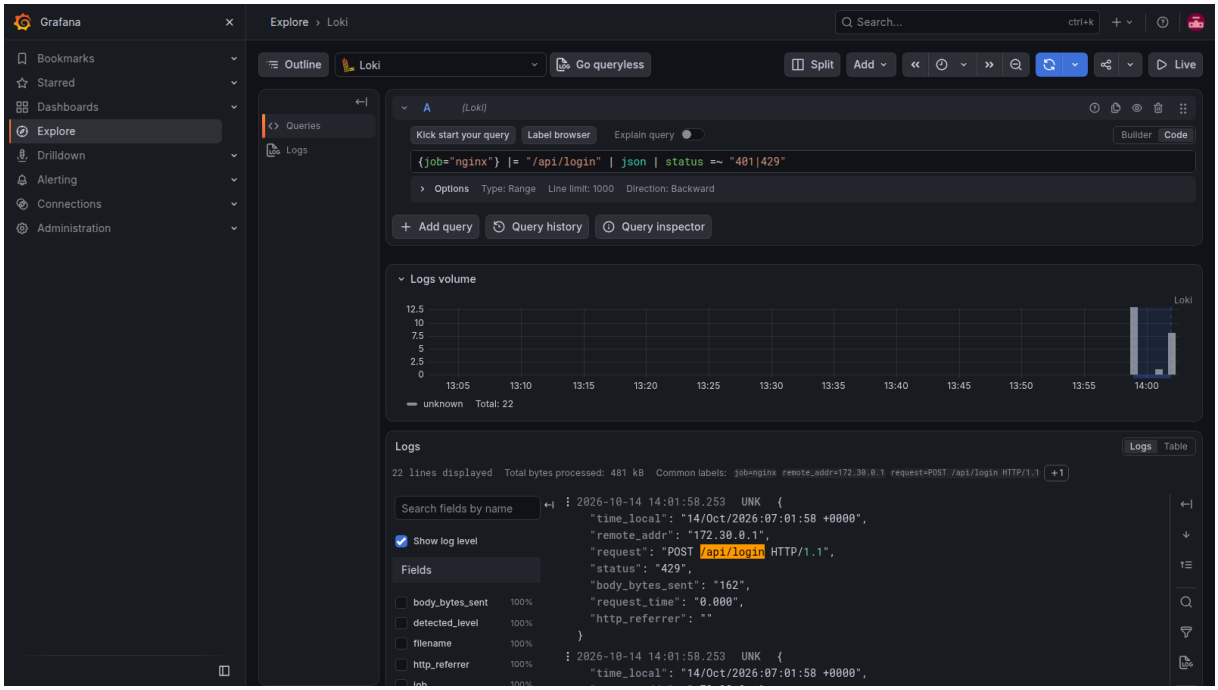
Hình 4.20. *Truy vấn 1 – toàn bộ log Nginx*



Hình 4.21. *Truy vấn 2 – lọc request có mã trạng thái từ 400 trở lên*



Hình 4.22. *Truy vấn 3 – log nghiệp vụ mượn sách*



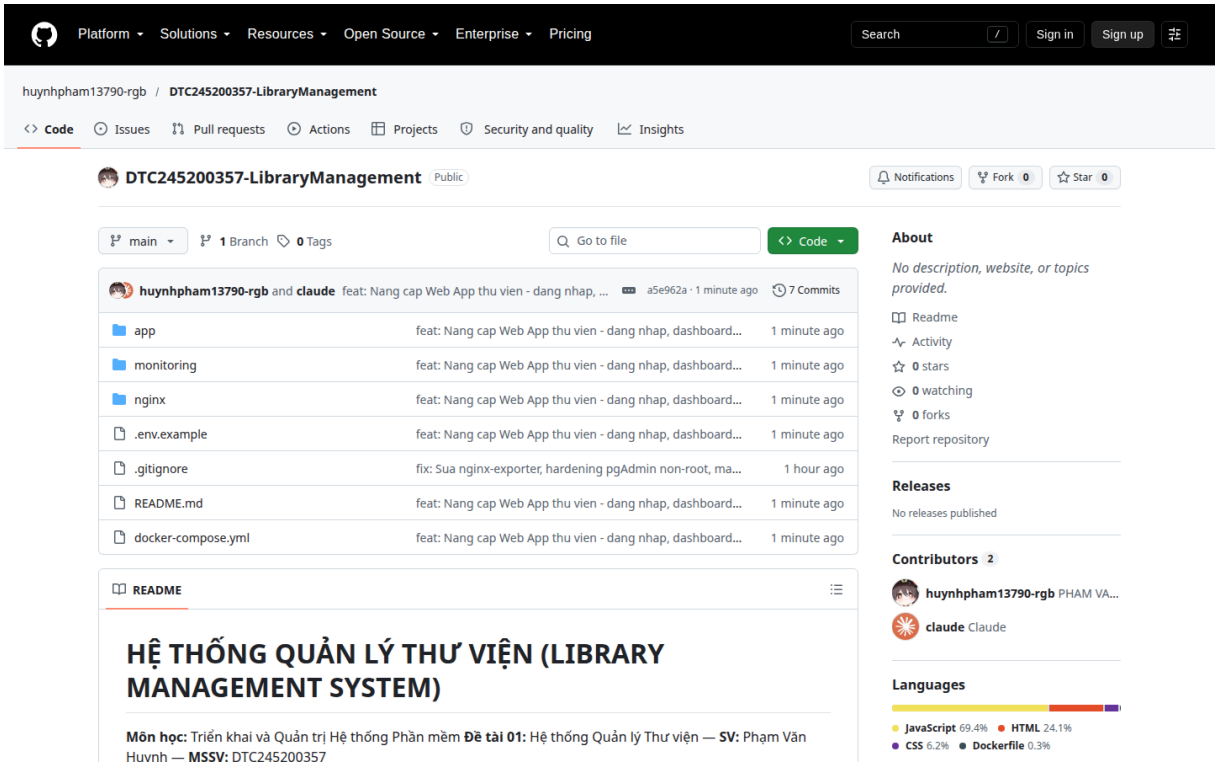
Hình 4.23. *Truy vấn 4 – các lần đăng nhập sai và bị chặn*

4.9. Quản lý mã nguồn trên GitHub

Dự án được quản lý trên GitHub với các commit theo từng giai đoạn (Hình 4.24): commit 1 triển khai ứng dụng, cơ sở dữ liệu, pgAdmin và Nginx; commit 2 tích hợp Prometheus, Grafana; commit 3 triển khai Loki, Promtail và hardening; các commit sau sửa lỗi cấu hình và nâng cấp chức năng.



Hình 4.24. *Lịch sử commit của dự án*



Hình 4.25. *Repository trên GitHub*

4.10. Kiểm thử

Bảng 4.3. *Kết quả kiểm thử chức năng và bảo mật*

STT	Kịch bản	Kết quả mong đợi	Kết quả
1	Gọi API khi chưa đăng nhập	HTTP 401	Đạt
2	Đăng nhập sai mật khẩu	Thông báo lỗi, HTTP 401	Đạt
3	Gửi liên tục yêu cầu đăng nhập	Nginx trả HTTP 429	Đạt
4	Thêm sách với số lượng âm	Từ chối, báo "Số lượng không hợp lệ"	Đạt

STT	Kịch bản	Kết quả mong đợi	Kết quả
5	Xóa sách đã có lịch sử mượn	Từ chối, HTTP 409	Đạt
6	Mượn sách đã hết bản	Từ chối "Sách đã được mượn hết"	Đạt
7	Độc giả có sách quá hạn mượn tiếp	Từ chối, báo số sách quá hạn	Đạt
8	Gia hạn phiếu lần thứ hai	Từ chối "chỉ được gia hạn 1 lần"	Đạt
9	Trả sách trễ hạn	Tự tính phạt 5.000đ/ngày	Đạt
10	Mã thẻ chứa ký tự đặc biệt	Từ chối, báo định dạng không hợp lệ	Đạt
11	Tên sách chứa thẻ HTML `<b>`	Hiển thị nguyên văn, không thực thi (chống XSS)	Đạt
12	Truy cập `https://localhost/metrics`	HTTP 404	Đạt
13	Truy cập `http://localhost`	Chuyển hướng 301 sang HTTPS	Đạt
14	Khởi động lại container ứng dụng	Dữ liệu còn nguyên, healthcheck *healthy*	Đạt

4.11. Đánh giá theo tiêu chí của đề bài

Bảng 4.4. *Tự đánh giá mức độ hoàn thành*

STT	Tiêu chí	Minh chứng
1	Quản lý mã nguồn GitHub	Hình 4.24, 4.25; README hướng dẫn chạy
2	Ứng dụng + Database	Hình 4.3–4.13, 4.15
3	Nginx reverse proxy	Hình 4.16, 4.17; truy cập web, pgAdmin, Grafana qua Nginx
4	Prometheus + Grafana	Hình 4.18, 4.19
5	Loki + LogQL	Bảng 4.2, Hình 4.20–4.23
6	Hardening	Bảng 3.4, Hình 4.17
7	Tổng thể	Hình 4.1: 11 dịch vụ chạy bằng Docker Compose

KẾT LUẬN

Kết quả đạt được

Báo cáo đã hoàn thành đầy đủ các yêu cầu của Đề tài 01:

- Xây dựng ứng dụng web quản lý thư viện với các chức năng đăng nhập và phân quyền, trang tổng quan thống kê, quản lý kho sách, quản lý độc giả, mượn – trả – gia hạn, tính tiền phạt, theo dõi sách quá hạn và quản lý tài khoản thủ thư.
- Triển khai 11 dịch vụ bằng Docker Compose: ứng dụng, PostgreSQL, pgAdmin, Nginx, Prometheus, Grafana, Loki, Promtail và ba exporter.
- Nginx làm reverse proxy duy nhất với HTTPS tự ký, các security header và giới hạn tần suất đăng nhập.
- Giám sát container, web server, cơ sở dữ liệu và các thao tác nghiệp vụ qua dashboard Grafana được quản lý bằng mã nguồn.
- Thu thập log tập trung bằng Loki + Promtail và truy vấn bằng 4 câu LogQL.
- Áp dụng 10 biện pháp hardening, trong đó có non-root container, cô lập mạng, mật khẩu mạnh, hạn chế quyền và security header.
- Mã nguồn được quản lý trên GitHub với các commit rõ ràng và README hướng dẫn chạy.

Qua quá trình thực hiện, em đã rút ra một số kinh nghiệm thực tế: cấu hình giám sát cần được kiểm tra bằng số liệu thật (exporter có thể báo UP nhưng không lấy được dữ liệu); mật khẩu chứa ký tự đặc biệt có thể làm hỏng chuỗi kết nối dạng URL; Docker không tự tạo lại mạng đã tồn tại khi thay đổi cấu hình; và một biện pháp bảo mật (chạy non-root) có thể kéo theo thay đổi ở thành phần khác (cổng của pgAdmin).

Hạn chế

- Chứng chỉ SSL là chứng chỉ tự ký nên trình duyệt vẫn cảnh báo.
- Phiên đăng nhập lưu trong bộ nhớ của ứng dụng, sẽ mất khi khởi động lại container và chưa hỗ trợ chạy nhiều bản sao.
- Promtail mới thu thập log của Nginx, chưa thu thập log của ứng dụng và các container khác.
- Chưa cấu hình cảnh báo (alerting) và chưa có cơ chế sao lưu cơ sở dữ liệu tự động.

Hướng phát triển

- Sử dụng chứng chỉ Let's Encrypt khi triển khai với tên miền thật.
- Lưu phiên đăng nhập trong Redis, mở rộng ứng dụng thành nhiều bản sao phía sau Nginx.
- Thu thập log toàn bộ container và cấu hình Alertmanager / Grafana Alerting gửi cảnh báo qua email hoặc Telegram.
- Sao lưu PostgreSQL định kỳ, bổ sung CI/CD với GitHub Actions và quét lỗ hổng

image.

- Bổ sung chức năng cho độc giả tự tra cứu, đặt trước sách và nhận thông báo nhắc hạn trả.

TÀI LIỆU THAM KHẢO

[1] Docker Inc. “Docker documentation,” Accessed: Oct. 14, 2026. [Online]. Available: https://docs.docker.com/

[2] Docker Inc. “Docker compose documentation,” Accessed: Oct. 14, 2026. [Online]. Available: https://docs.docker.com/compose/

[3] F5, Inc. “Nginx documentation – reverse proxy, ssl termination and rate limiting,” Accessed: Oct. 14, 2026. [Online]. Available: https://nginx.org/en/docs/

[4] The PostgreSQL Global Development Group. “Postgresql 16 documentation,” Accessed: Oct. 14, 2026. [Online]. Available: https://www.postgresql.org/docs/16/

[5] Prometheus Authors. “Prometheus documentation,” Accessed: Oct. 14, 2026. [Online]. Available: https://prometheus.io/docs/

[6] Grafana Labs. “Grafana documentation – provisioning and dashboards,” Accessed: Oct. 14, 2026. [Online]. Available: https://grafana.com/docs/grafana/latest/

[7] Grafana Labs. “Grafana loki documentation – logql,” Accessed: Oct. 14, 2026. [Online]. Available: https://grafana.com/docs/loki/latest/query/

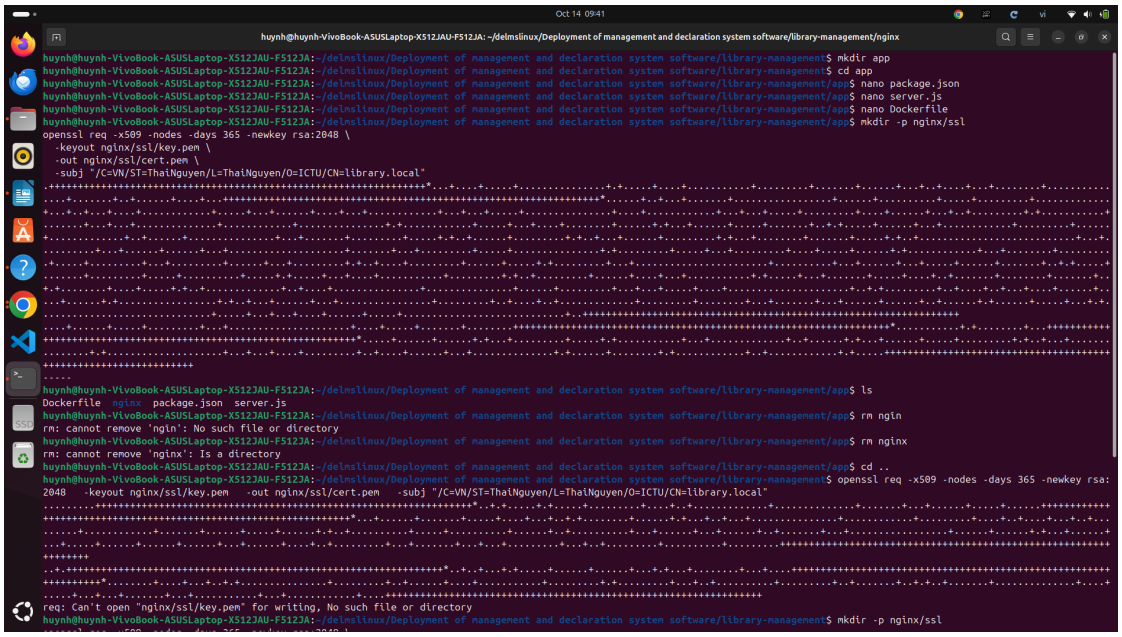
[8] *Cis docker benchmark*, Center for Internet Security, 2023. [Online]. Available: https://www.cisecurity.org/benchmark/docker

[9] OWASP Foundation. “Owasp top 10 – the ten most critical web application security risks,” Accessed: Oct. 14, 2026. [Online]. Available: https://owasp.org/Top10/

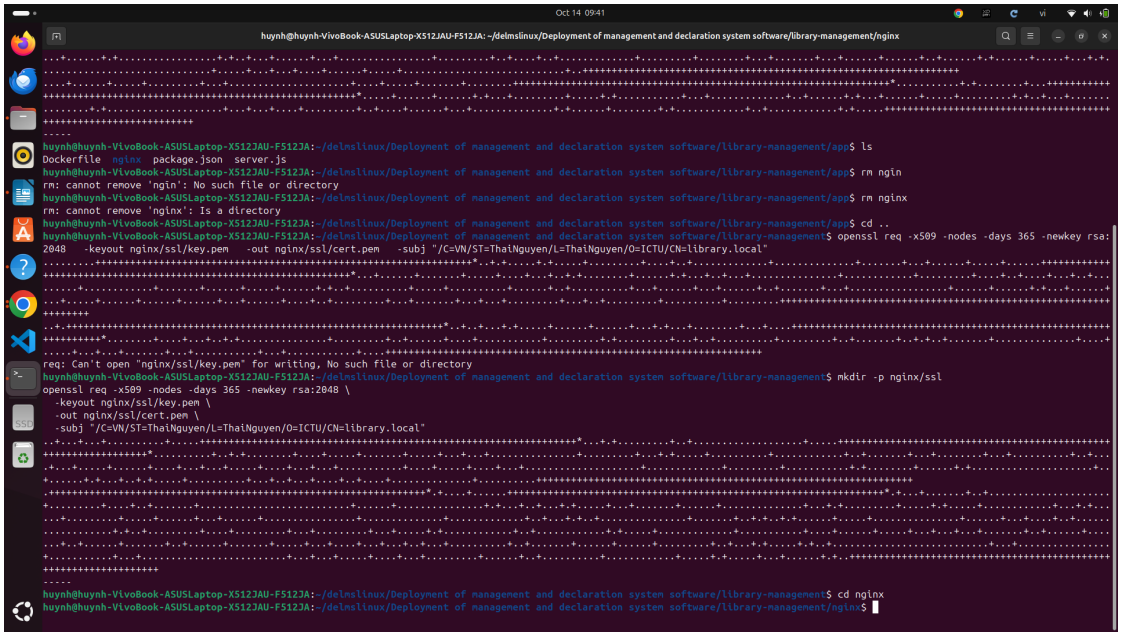
[10] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

PHỤ LỤC: HÌNH ẢNH QUÁ TRÌNH THỰC HIỆN

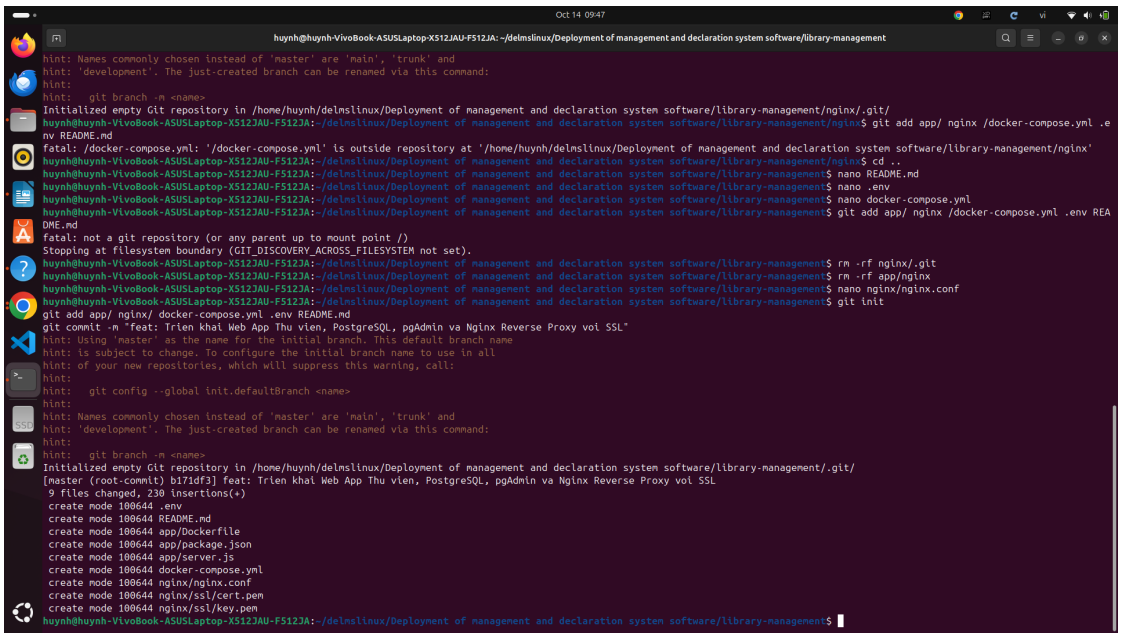
Phụ lục tổng hợp một số ảnh chụp màn hình trong quá trình em thực hiện triển khai hệ thống trên máy cá nhân, theo thứ tự thời gian.



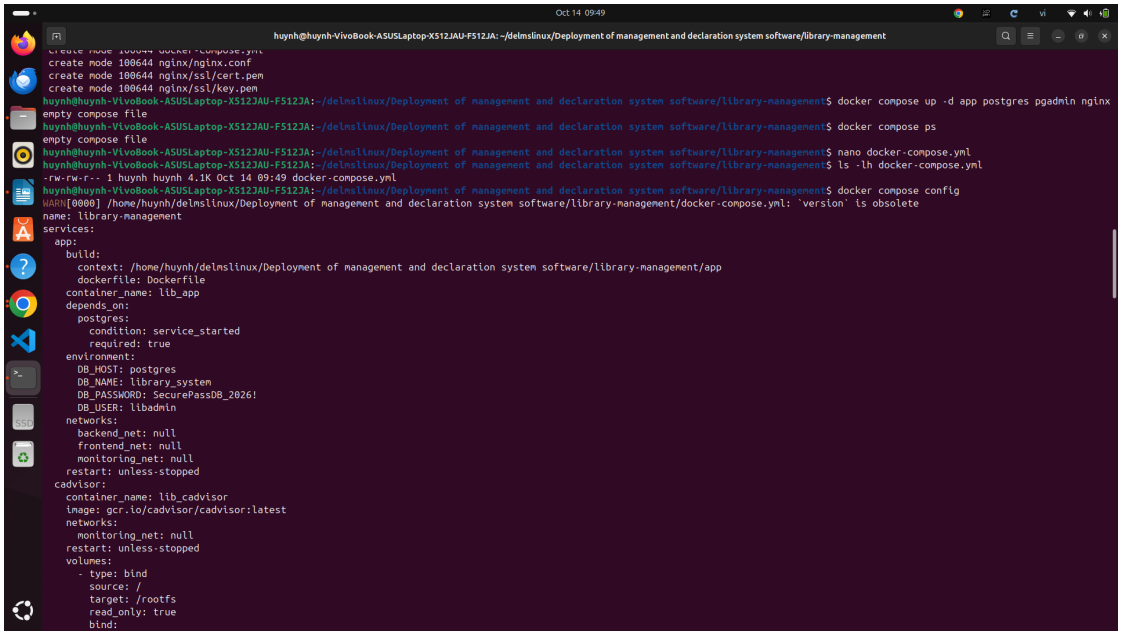
Hình PL.1. *Tạo cấu trúc thư mục dự án và các file mã nguồn ứng dụng*



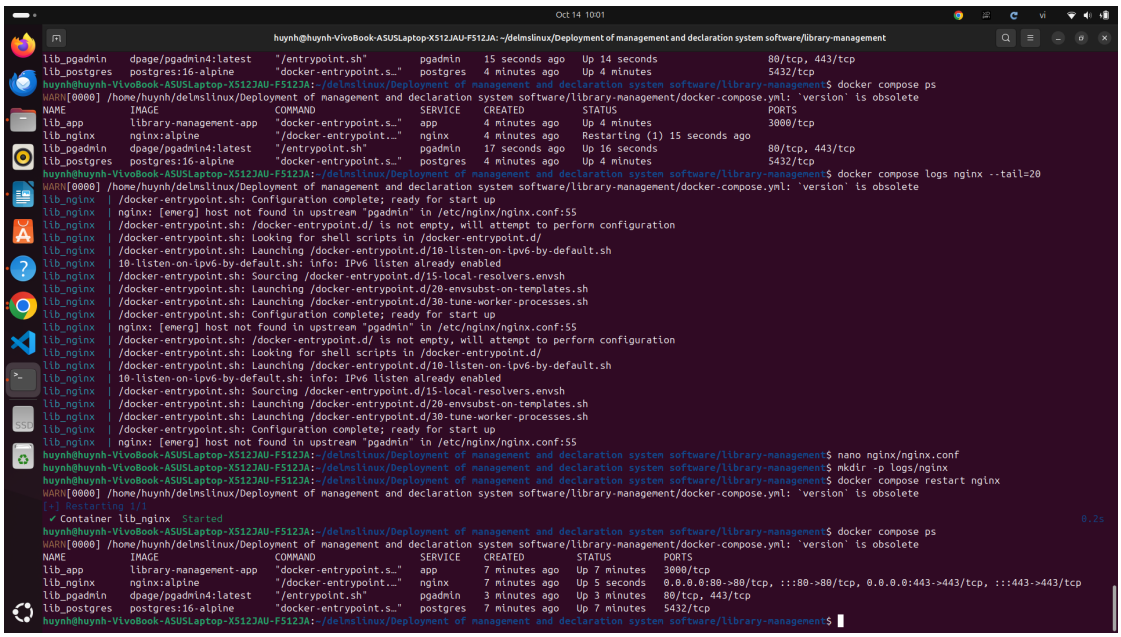
Hình PL.2. *Tạo chứng chỉ SSL tự ký bằng OpenSSL*



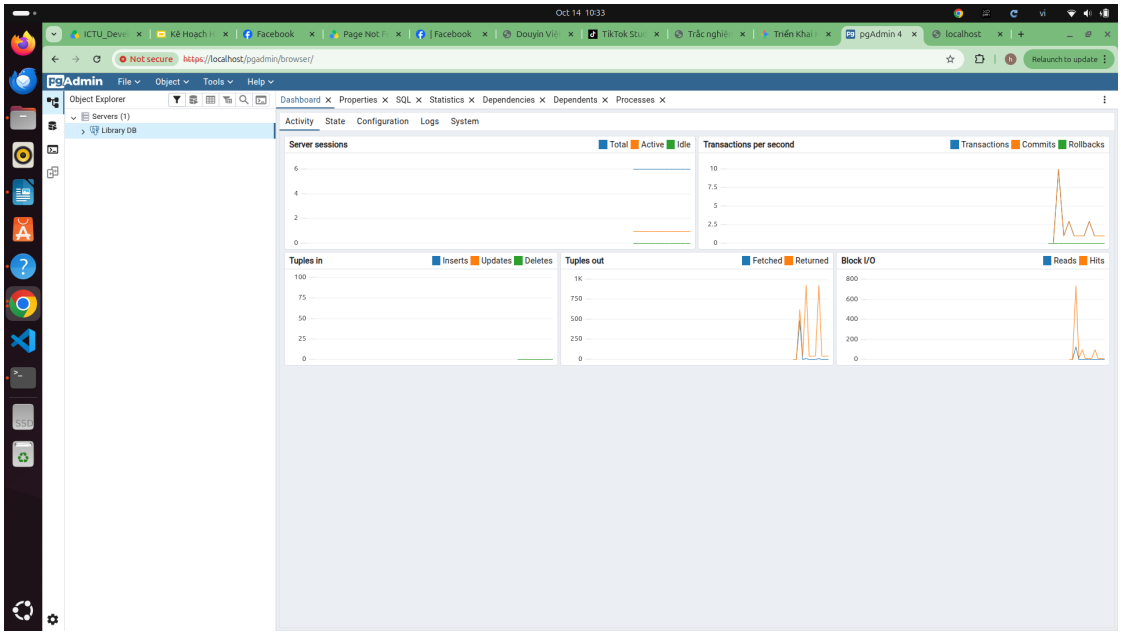
Hình PL.3. *Khởi tạo Git repository và commit đầu tiên*



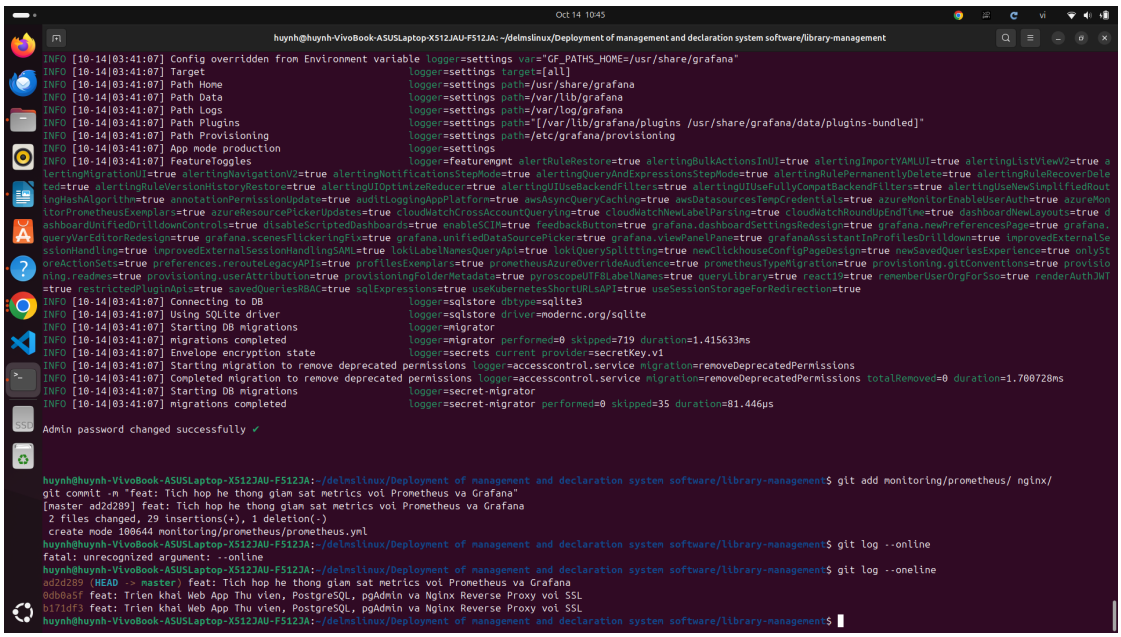
Hình PL.4. *Kiểm tra cấu hình hệ thống bằng lệnh docker compose config*



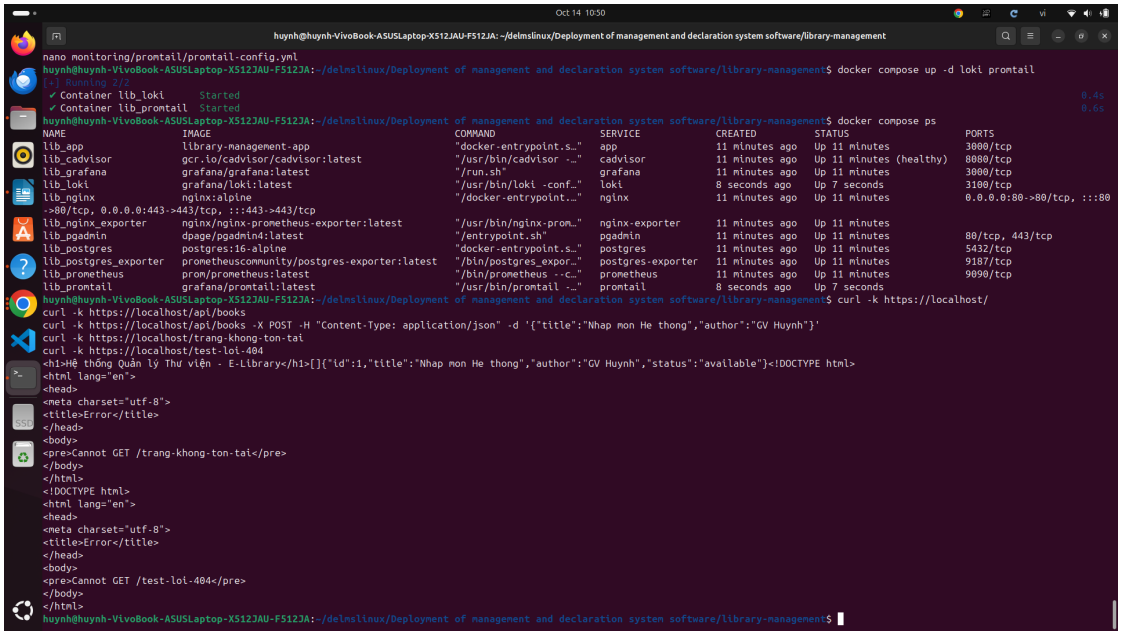
Hình PL.5. *Kiểm tra trạng thái container và đọc log Nginx khi gỡ lỗi kết nối tới pgAdmin*



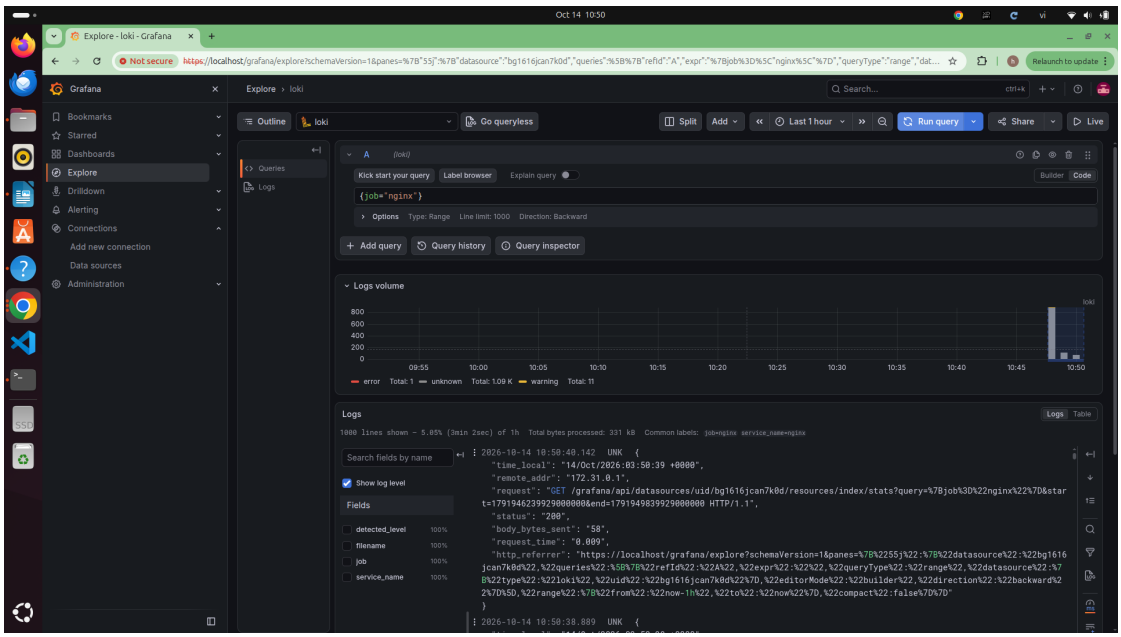
Hình PL.6. *Giao diện pgAdmin theo dõi hoạt động của PostgreSQL*



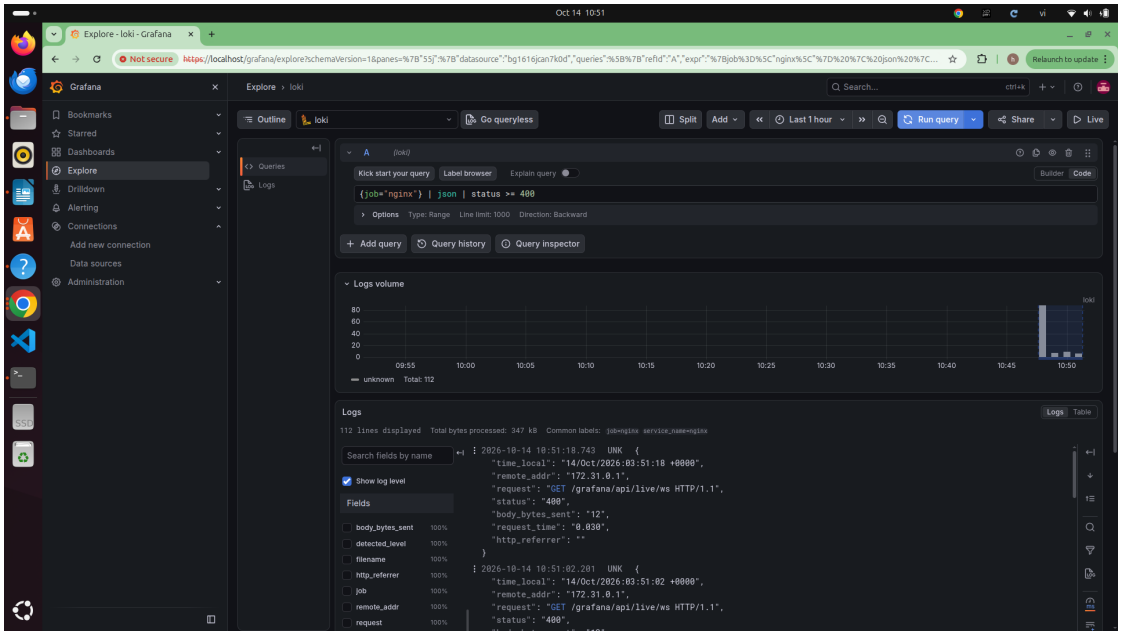
Hình PL.7. *Log khởi động Grafana và commit tích hợp Prometheus, Grafana*



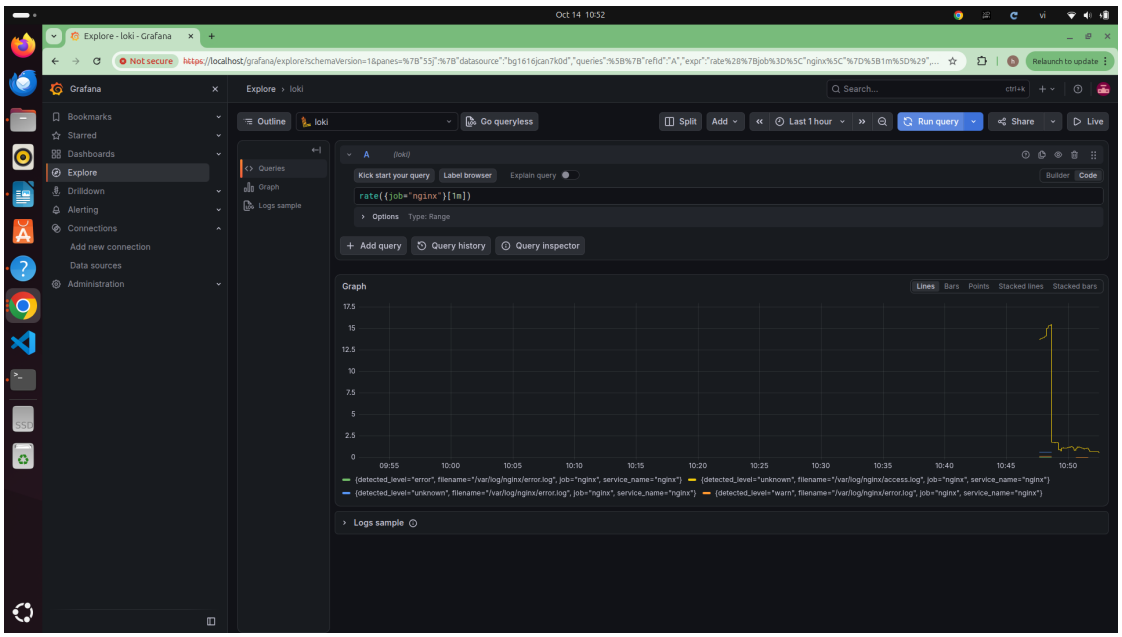
Hình PL.8. *Kiểm tra các container và gọi thử API ứng dụng bằng curl*



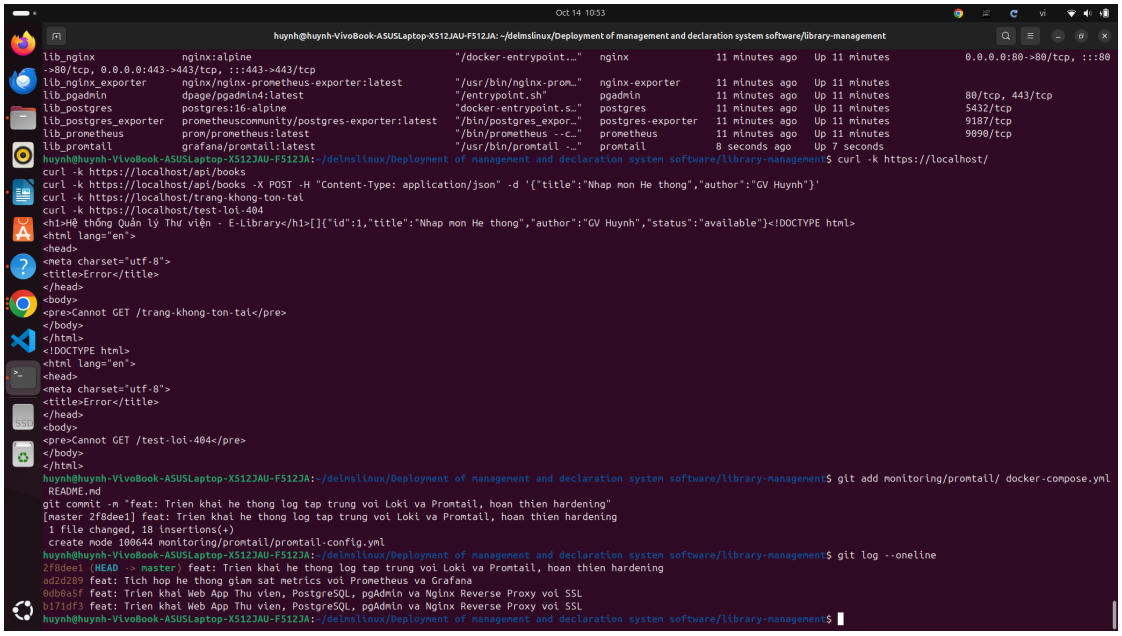
Hình PL.9. *Truy vấn log Nginx trên Grafana Explore (lần chạy đầu)*



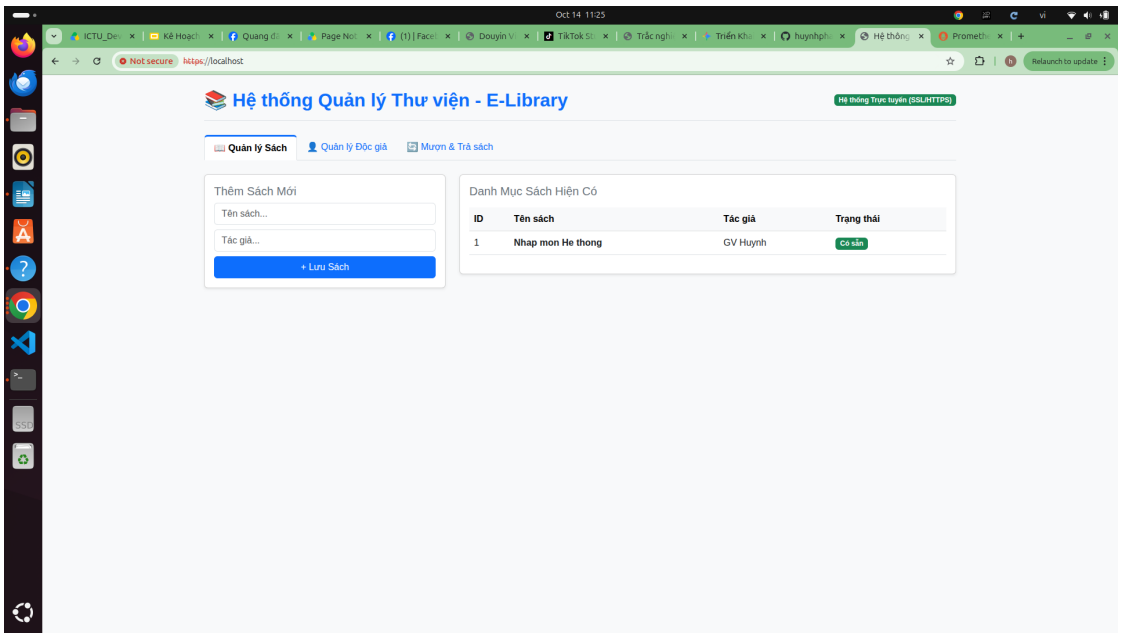
Hình PL.10. *Lọc log theo mã trạng thái trên Grafana Explore (lần chạy đầu)*



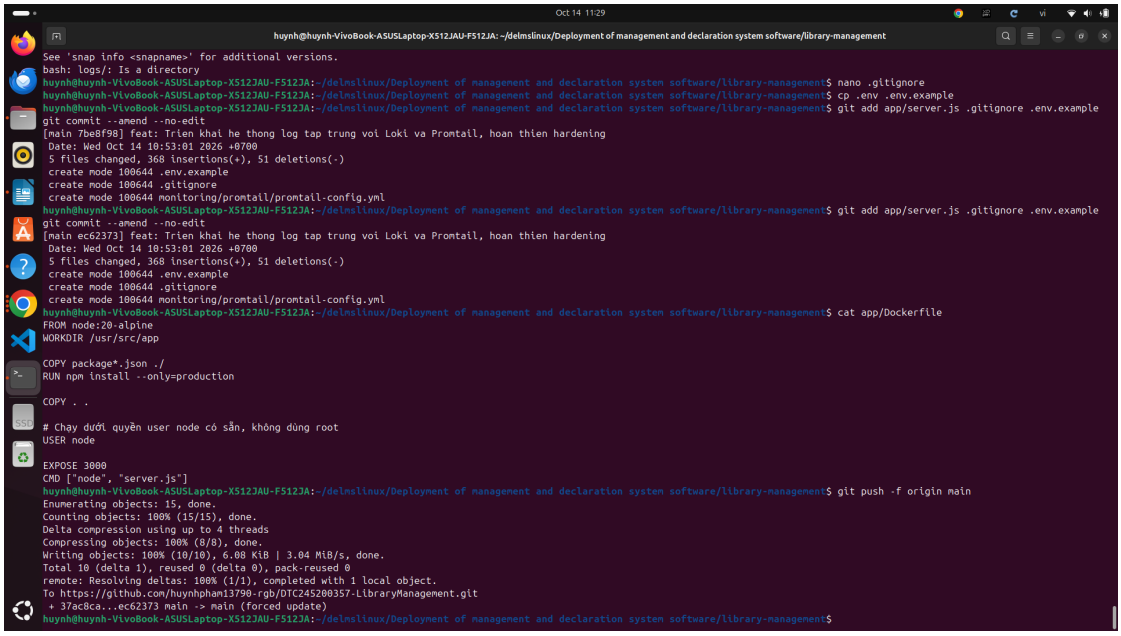
Hình PL.11. *Truy vấn dạng metric từ log trên Grafana Explore*



Hình PL.12. *Kiểm thử API và commit triển khai Loki, Promtail*



Hình PL.13. *Giao diện phiên bản đầu tiên của ứng dụng*



Hình PL.14. *Hoàn thiện cấu hình và đẩy mã nguồn lên GitHub*

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Thái Nguyên, tháng … năm …

Giáo viên hướng dẫn